

TOPIC 1 INTRODUCTION TO WEB DEVELOPMENT

TOPIC 2 HTML BASICS

CHAPTER ONE: INTRODUCTION TO WEB DEVELOPMENT

1. THE INTERNET

The **Internet** is a vast global network of computers and devices connected to each other, allowing communication and the sharing of resources. It is the foundation for modern digital communication, enabling users to access websites, send emails, transfer files, and much more.

The Internet is a **network of networks**, where various private, public, academic, and governmental networks communicate with one another using standardized protocols.



2. History of the Internet

The origin of the Internet dates back to the late 1960s, with the development of **ARPANET**, a project funded by the U.S. Department of Defense. The primary goal was to create a network that would allow computers in different locations to communicate over long distances.

Key milestones in the development of the Internet:

- **1969:** ARPANET's first message sent.
- **1980s:** Development of protocols like **TCP/IP**, allowing for a standardized way for different networks to communicate.
- **1990:** Tim Berners-Lee developed the **World Wide Web (WWW)**, introducing a system of hypertext documents that could be accessed via the Internet.
- **1993:** The first graphical web browser, **Mosaic**, was introduced, making the web more user-friendly and visually appealing.

3. How Does the Internet Work?

The Internet works by connecting computers to each other through a combination of **hardware** (like routers, switches, and cables) and **software** (protocols that manage data transfer).

IP Addresses

Every device connected to the Internet has a unique identifier called an **IP address** (Internet Protocol address). An IP address helps in identifying a device and its location within the network.

There are two types of IP addresses:

- **IPv4:** Most common format (e.g., 192.168.1.1).
- **IPv6:** A newer format designed to handle more devices (e.g., 2001:0db8:85a3::8a2e:0370:7334).

DNS (Domain Name System)

The **Domain Name System (DNS)** translates domain names (like www.google.com) into IP addresses. This allows humans to use readable domain names instead of memorizing complex IP addresses.



Data Transfer: Packets

When data is sent over the Internet, it is broken down into smaller chunks called **packets**. These packets travel independently through the network and are reassembled at the destination.

4. THE WORLD WIDE WEB (WWW)

The **World Wide Web (WWW)** is an information system that allows users to access hypertext documents (web pages) through the Internet. While the Internet is the underlying network, the web is one of the services that run on top of it.

Key components of the web:

- **Web Pages:** Documents containing text, images, videos, and links to other web pages.
- **Hypertext Markup Language (HTML):** The language used to structure and display web pages.
- **Hyperlinks:** Clickable links that allow users to navigate between web pages.

Web Pages:

A **web page** is a digital document that can be viewed in a web browser, containing various types of content such as text, images, videos, and interactive elements. Every web page is identified by a unique URL (Uniform Resource Locator), which acts as its address on the web.

Key features of web pages include:

- **Text:** The primary content that conveys information.
- **Images:** Visual elements that enhance the page's appearance and provide additional context.
- **Videos:** Multimedia content embedded within the page to deliver rich, dynamic experiences.
- **Links:** Hyperlinks that connect the page to other pages or resources, enabling users to navigate between different parts of the web.

Web pages can be **static** or **dynamic**:

- **Static web pages** are pre-built and remain the same until manually updated.
- **Dynamic web pages** are generated in real-time based on user interactions or data retrieved from a server.

HYPERTEXT MARKUP LANGUAGE (HTML):

HTML is the standard language used to create and structure content on the web. It defines the elements that form the layout and content of a web page, like headers, paragraphs, images, and links.

HTML is made up of **tags** that describe different types of content. These tags are enclosed in angle brackets (< >) and are typically paired (opening and closing tags). For example

```
<!DOCTYPE html>
<html>
<head>
  <title>Working with Hyperlinks</title>
</head>
<body>
  <h1>Welcome to My Web Page</h1>
  <p>This is an example of a web page using HTML.</p>
  
</body>
</html>
```

Hyperlinks

A **hyperlink** (or simply "link") is a clickable element on a web page that allows users to navigate to other web pages or resources. Hyperlinks can be embedded in text, images, or buttons.

In HTML, a hyperlink is created using the <a> (anchor) tag. The href attribute specifies the URL or destination the link points to. For example:

```
<!DOCTYPE html>

<html>
<head>
  <title>Sample Web Page</title>
</head>
<body>
  <h1>Welcome to My Web Page</h1>
  <p>This is an example of a web page using HTML.</p>
  
  <a href="info.html">Click for more information</a>
</body>
</html>
```

Welcome to My Web Page

This is an example of a web page using hyperlinks.



[Click for more information](#)

There are two main types of hyperlinks:

1. **Internal links:** These link to another page within the same website.

```
<a href="about.html">About Us</a>
```

2. **External links:** These link to a different website.

```
<a href="https://www.seku.ac.ke">Visit SEKU home page</a>
```

Hyperlinks are an essential part of the web, as they enable users to move between different resources and navigate the vast network of information on the Internet.

5. How the Web Works: HTTP and HTTPS

The web operates using a protocol called **HTTP (Hypertext Transfer Protocol)**. This protocol defines how data is exchanged between a user's browser and the web server that hosts the website.

- **HTTP:** A standard protocol for transferring web pages.
- **HTTPS:** A secure version of HTTP, where the "S" stands for "Secure." It uses encryption (usually via SSL/TLS) to ensure data privacy and integrity.

Differences between HTTP and HTTPS

1. Security:

- **HTTP:** Data transmitted using HTTP is **unencrypted**. This means that information sent between a browser and a web server, such as passwords or personal details, can be intercepted by malicious actors (e.g., during a "man-in-the-middle" attack).
- **HTTPS:** HTTPS uses **encryption** to secure the communication between the browser and the server, ensuring data privacy. This makes it much harder for unauthorized parties to eavesdrop on the communication.

2. Encryption:

- **HTTP:** No encryption is used in HTTP, meaning the data is sent as plain text, which can be easily read by anyone who intercepts it.
- **HTTPS:** HTTPS uses **SSL/TLS (Secure Sockets Layer/Transport Layer Security)** to encrypt the data. SSL/TLS creates a secure, encrypted channel that protects the integrity and confidentiality of the data being transmitted.

3. Data Integrity:

- **HTTP:** Since there is no encryption, the data sent through HTTP can be modified or tampered with by attackers without the user or server realizing it.
- **HTTPS:** In HTTPS, the SSL/TLS protocol ensures that data cannot be altered during transmission. Any tampering with data is detected and the communication is dropped, protecting the integrity of the information.

4. Authentication:

- **HTTP:** With HTTP, there is no guarantee that the website you're communicating with is legitimate. Attackers can create fake websites that look authentic to steal information.
- **HTTPS:** HTTPS uses **digital certificates** issued by trusted Certificate Authorities (CAs) to authenticate the website. This ensures the user is communicating with the legitimate site and not a fake one. Websites using HTTPS typically display a padlock icon in the address bar, confirming their security.

5. Port Usage:

- **HTTP:** Uses **port 80** by default for communication.
- **HTTPS:** Uses **port 443** by default for secure communication.

6. Performance:

- **HTTP:** Since HTTP does not involve encryption or additional security steps, it is slightly faster than HTTPS.
- **HTTPS:** Due to the encryption and decryption process, HTTPS can introduce a small performance overhead. However, with modern optimizations and hardware, the difference in speed is minimal.

7. SEO and Trust:

- **HTTP:** Websites using HTTP are often flagged as "not secure" by modern web browsers, especially if they handle sensitive data (e.g., login pages, payment gateways). This can negatively impact a site's reputation.
- **HTTPS:** Search engines like Google prefer HTTPS websites and rank them higher in search results. Additionally, HTTPS provides a sense of trust and security to users, which can improve user engagement and conversions.

Summary Table:

Feature	HTTP	HTTPS
Security	No security, data sent in plain text	Secured by SSL/TLS encryption
Encryption	None	Data is encrypted during transfer
Data Integrity	Data can be modified or tampered	Data is protected from tampering
Authentication	No authentication	Authenticates websites using certificates
Port	Port 80	Port 443
Speed	Slightly faster	Slightly slower due to encryption
SEO	No SEO benefits	Better SEO rankings

Web Browsers

A **web browser** is software that allows users to access the web. Examples include:

- **Google Chrome**
- **Mozilla Firefox**
- **Safari**

- **Microsoft Edge**



The browser sends a request to the web server, retrieves the web page, and displays it to the user.

Web Servers

A **web server** is a computer system that hosts websites and serves web pages to users when requested. Popular web servers include:

- **Apache**
- **Nginx**
- **Microsoft IIS**

6. Web Technologies: Frontend and Backend

Frontend Technologies

The **frontend** refers to the part of a website that users interact with. It involves designing the layout, styling, and overall user experience.

Key frontend technologies:

- **HTML (Hypertext Markup Language):** The backbone of a web page, defining its structure and content.
- **CSS (Cascading Style Sheets):** Used to style the HTML, controlling layout, fonts, colors, etc.
- **JavaScript:** A programming language used to add interactivity and dynamic behavior to web pages.

Example of HTML:

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Web Page</title>
</head>
<body>
  <h1>Welcome to My Website</h1>
  <p>This is a simple HTML document.</p>
</body>
</html>
```

Backend Technologies

The **backend** refers to the server-side of a website, where data is processed and stored. This includes databases, server-side programming, and APIs.

Key backend components:

- **Databases:** Where the data is stored (e.g., MySQL, MongoDB).
- **Server-Side Languages:** Programming languages that handle the logic behind a website (e.g., Python, PHP, Node.js, Ruby).
- **APIs (Application Programming Interfaces):** These allow communication between the frontend and backend, enabling dynamic functionality like form submissions or fetching data from external services.

Client-Server Architecture

In a **client-server model**, the **client** is typically a user's device or an application that requests services, data, or resources. The **server** is the system that provides those resources or services. The two interact via a network, often the internet or a local area network (LAN).

Client

- A client can be a computer, smartphone, or any device connected to a network.
- It sends a request to the server for a particular service, such as requesting a webpage or data from a database.
- Clients can be web browsers, email clients, or specialized software like banking apps.

Server

- A server is typically a powerful machine or cloud-based system that processes client requests and provides the requested data or services.
- Servers can host websites, databases, email services, file storage, and more.
- The server often performs tasks such as managing access, storing data, or providing results after complex calculations.

How does it work?

Here's a basic flow of how clients and servers interact:

1. **Client Request:** The client sends a request to the server. For example, when you type a website URL into your browser, your browser acts as the client and sends a request to the server hosting the website.

2. **Server Processing:** The server receives the request and processes it. If the request is to retrieve a webpage, the server will locate the page and prepare it for transmission.
3. **Response Sent:** The server sends the requested data back to the client. In our web example, the server sends the HTML file and other assets (such as images and stylesheets) back to the web browser.
4. **Client Display:** The client, which could be a web browser, processes the data and presents it to the user, such as displaying the webpage.

Features of Client-Server Architecture

- **Decentralization:** In client-server architecture, the server acts as a centralized point for services, but clients can be decentralized, existing anywhere on the network.
- **Scalability:** Servers can be scaled to handle increasing numbers of clients, either by adding more powerful hardware or distributing the load across multiple servers.
- **Security:** Since all data and services are centralized on the server, managing security policies and protecting sensitive information becomes easier

Types of Servers in Client-Server Architecture

- **Web Server:** A web server handles requests for web pages and other assets. For example, when you access a website, the server processes the request and serves the website's content (HTML, CSS, JavaScript, etc.).
- **Database Server:** A database server manages data storage and retrieval for clients, often using database management systems (DBMS) like MySQL or PostgreSQL. The server executes SQL queries requested by the client.
- **File Server:** A file server stores and manages access to files. Users on a network can request files from the server, which will either allow them to download or access those files as needed.
- **Application Server:** This type of server hosts and runs applications. When a client requests certain data or processing from an application, the application server processes the request and returns the output.



Technologies Used in Web Servers

Web servers are essential for hosting websites, serving content, and enabling communication between clients and servers over the internet. Several technologies are involved in the functioning of web servers, including software, protocols, and infrastructure components.

1. Web Server Software

The web server software is the backbone of any web server. It processes client requests, serves web pages, and manages communication between the client and the server.

- **Apache HTTP Server:** One of the most widely used open-source web server software. It supports multiple programming languages, dynamic content, and is highly customizable with modules.
- **Nginx:** Known for its high performance, scalability, and ability to handle a large number of concurrent connections. It is often used for load balancing, reverse proxying, and as a static content server.
- **Microsoft Internet Information Services (IIS):** A web server software created by Microsoft for use with Windows Server. IIS integrates well with ASP.NET and other Microsoft technologies.
- **Lighttpd:** Lighttpd is an open-source web server optimized for speed-critical environments while remaining standards-compliant, secure and flexible.



2. Operating Systems

The choice of the operating system (OS) can impact the performance, security, and reliability of the web server.

- **Linux:** Often used with Apache and Nginx, Linux is the most popular OS for web servers due to its stability, security, and open-source nature. Common distributions include Ubuntu, CentOS, and Debian.
- **Windows Server:** Commonly used with IIS, Windows Server is known for its integration with other Microsoft products. It supports a wide range of technologies, including ASP.NET and Microsoft SQL Server.
- **Unix:** While less commonly used than Linux, Unix-based systems such as FreeBSD are also employed as web server operating systems due to their reliability.

4. Scripting and Programming Languages

Web servers interact with scripts and programming languages to process dynamic content and generate webpages in real-time.

- PHP: A server-side scripting language commonly used with Apache and Nginx to generate dynamic web content. It's widely used in web applications like WordPress.
- Node.js: A runtime environment that allows developers to use JavaScript on the server side. It's popular for building scalable, high-performance applications, especially for real-time applications like chat or gaming.
- Python: Used with frameworks such as Django or Flask, Python is often employed in web server environments for applications that require complex data processing or machine learning integration.
- Ruby: Ruby on Rails is a popular web framework used with web servers for rapid application development, especially in startups and agile environments.
- ASP.NET: A web framework from Microsoft for building dynamic web applications, commonly used with IIS and Windows Server.

5. Databases

Web servers often interact with databases to serve dynamic content, such as user data, blog posts, and e-commerce information.

- MySQL: An open-source relational database management system (RDBMS) often used with PHP and other web technologies for storing structured data.
- PostgreSQL: A powerful, open-source RDBMS that supports advanced features, including JSON, full-text search, and geographic data.
- MongoDB: A NoSQL database that stores data in a flexible, document-oriented format, often used in modern web applications requiring scalability and performance.
- Microsoft SQL Server: A relational database management system developed by Microsoft, commonly used with IIS and ASP.NET applications.

CHAPTER TWO: HTML BASICS

HTML, which stands for Hypertext Markup Language, is the language used to describe structured documents and the language used to create web pages on the internet. You can use any text editor to write the HTML code, such as Notepad. HTML tags are the keywords on a web page that define how your web browser must format and display your web page. Almost all tags contain two parts, an opening, and a closing tag. For example, `<html>` is the opening tag and `</html>` is the closing tag. Note that the closing tag has the same text as the opening tag, but has an additional forward-slash (/) character.

Basic HTML Tags

Head Tag

The head tag `<head>` contains all the elements describing the document.

Title Tag

The title tag `<title>` specifies the HTML page title, which is shown in the browser's title bar.

Body Tag

The body tag `<body>` is where you insert your web page's content.

Paragraph Tag

A paragraph tag `<p>` is used to define a paragraph on a web page.

Heading Tag

The HTML heading tag is used to define the heading of the HTML document. The `<h1>` tag defines the most important tag, and `<h6>` defines the least.

STANDARD HTML DOCUMENT STRUCTURE FORMAT

A correct HTML document must always include certain structural tags. These tags are `<HTML>`, `<HEAD>`, `<TITLE>` and `<BODY>`

The four tags are the main tags that form the basic structure of HTML document

```

<HTML>
  <HEAD>
    <TITLE>    </TITLE>
  </HEAD>
<BODY>

</BODY>
</HTML>

```

More detailed structure of an HTML document:

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Basic HTML Document</title>
  </head>
  <body>

  </body>
</html>

```

Explanation of the Structure:

1. `<!DOCTYPE html>`: This declaration defines the document type and version of HTML (HTML5 in this case).
2. `<html lang="en">`: The root element of the HTML document with a language attribute set to English.
3. `<head>`: Contains meta-information about the document, such as character set, viewport settings, title, and links to stylesheets and scripts.
 - `<meta charset="UTF-8">`: Specifies the character encoding for the document.
 - `<meta name="viewport" content="width=device-width, initial-scale=1.0">`: Ensures proper rendering and touch zooming on mobile devices.
 - `<title>`: Sets the title of the document, which appears in the browser tab.
4. `<body>`: Contains the content of the document that will be displayed in the web browser.

This basic structure includes common elements used to organize and structure content on a webpage, following standard HTML5 practices.

Paragraph Tag

Paragraph tag offers a way to structure your text into different paragraphs. Each paragraph of text should go in between an opening and a closing tag as shown below in the example:

```
<html>
<head><title></title></head>
<body>
<p>First paragraph

<p>Second paragraph

</body>
</html>
```

Here is a first paragraph of text.

Here is a second paragraph of text.

Here is a third paragraph of text.

HTML FORMATTING

HTML Formatting is a process of formatting text for better look and feel. HTML provides us ability to format text. There are many formatting tags in HTML. These tags are used to make text bold, italicized, or underlined.

		This is a physical tag, which is used to bold the text written between it.
	<i>	This is a physical tag which is used to make text italic.
		This is a logical tag which is used to display content in italic
	<u>	This tag is used to underline text written between it.
		This is a logical tag, which tells the browser that the text is important.

The tag

The tag is an HTML element used to apply bold formatting to text content. It stands for "bold" and is a part of the set of inline elements in HTML. When you wrap text within tags, the enclosed text is rendered in a bold font style by web browsers.

```
<html>
```

```
<head></head>
```

```
<body>
```

Working with the `<b>BOLD</b>` tag

Working with HTML `<b>tags</b>`

```
</body>
```

```
</html>
```

Output:

Working with the **BOLD** tag

Working with HTML **tags**

The `<strong>` tag

The `<strong>` tag is an HTML element used to denote text with strong importance, emphasis, or relevance. It is one of the semantic markup elements in HTML designed to convey meaning beyond mere presentation.

When you enclose text within `<strong>` tags, it tells web browsers and assistive technologies that the enclosed text should be treated as having strong importance or emphasis.

The `<i>` HTML Tag

The `<i>` element is used to differentiate words from the surrounding text by styling the marked text in italics without implying any added emphasis to the italicized words.

The `<em>` HTML Tag

The `<em>` HTML element marks text that has stress emphasis. The `<em>` element can be nested, with each level of nesting indicating a greater degree of emphasis.

Difference between `<i>` and `<em>`

The `<em>` and `<i>` tags produce similar visual results. However, the semantic meaning is different. The `<em>` element represents stress emphasis of its contents, while the `<i>` element represents text that is set off from the normal prose, such as a foreign word, fictional character thoughts, or when the text refers to the definition of a word instead of representing its semantic meaning.

An example for `<em>` could be: "Work *smart* !". A person or software reading the text would pronounce the words in italics with an emphasis, using verbal stress.

An example for `<i>` could be: "The *Queen Mary* sailed last night". Here, there is no added emphasis or importance on the word "Queen Mary". It is merely indicated that the object in question is not a queen named Mary, but a ship named *Queen Mary*.

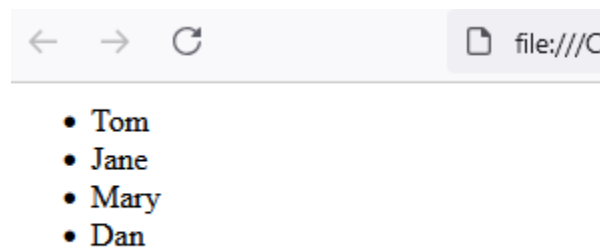
Ordered and unordered List

Unordered HTML List

An unordered list starts with the `<ul>` tag. Each list item starts with the `<li>` tag.

The list items will be marked with bullets (small black circles) by default:

```
<ul>
  <li>Tom</li>
  <li>Jane</li>
  <li>Mary</li>
  <li>Dan</li>
</ul>
```

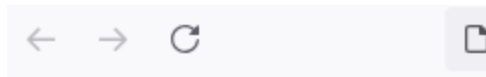


Ordered HTML List

An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag.

The list items will be marked with numbers by default:

```
<ol>
  <li>Tom</li>
  <li>Jane</li>
  <li>Mary</li>
  <li>Dan</li>
</ol>
```

1. Tom
2. Jane
3. Mary
4. Dan

WORKING WITH IMAGES

To display an image on your web page, you'll use the `<img>` element. It's a self-closing tag, which means you don't need a closing `</img>` tag. Instead, you place the image source and other attributes within the opening tag. Here's the basic syntax:

```

```

- The `src` attribute specifies the image file's location.
- The `alt` attribute provides alternative text for screen readers and that's displayed if the image fails to load

```
<html>
<head>
  <title>Working with Hyperlinks</title>
</head>
<body>
  <h1>Working with Images</h1>
  
</body>
</html>
```

Working with Images

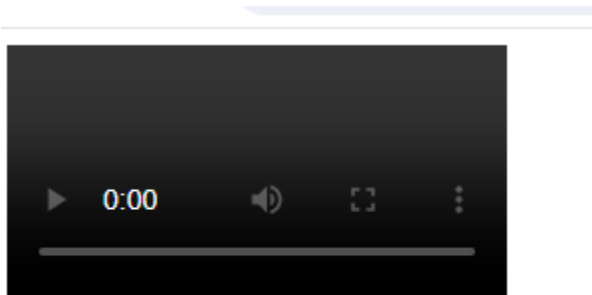


WORKING WITH VIDEO

The `<video>` [HTML](#) element embeds a media player which supports video playback into the document. You can use `<video>` for audio content as well, but the [audio](#) element may provide a more appropriate user experience.

Syntax:

```
<!DOCTYPE html>
<html>
<head></head>
<body>
<video controls width="250">
  <source src="https://www.youtube.com/watch?v=J1zPx2c7Do" type="video/mp4"/>
  ONLINE VIDEO
</video>
<p>Download the <a href="https://www.youtube.com/watch?v=J1zPx2c7Do">MP4</a>
video.</p>
</body>
</html>
```



Download the [MP4](#) video.

WORKING WITH ANCHOR TAG

Text Anchor

The primary purpose of an anchor tag is to link one page to another page or to a section of the same page. The anchor tag is also known as a HyperLink. Like any other HTML tags, you use the following construct to create an anchor tag with href attribute to link to another page.

```
<a href="index.html">Home</a>
```

You can use the anchor tag to link to an email address as shown below:

```
<a href="mailto:myemail@email.com">Send email</a>
```

Image Anchor

There comes a time when you want to use image as a link. To achieve that we use anchor tag. Example given below:

```
<a href="index.html"></a>
```

WORKING WITH TABLES

A table is a structured set of data made up of rows and columns (**tabular data**). A table allows you to quickly and easily look up values that indicate some. An intersection of row and column makes a cell. We enter data in a table cell.

Sno.	Regno	First name	Surname
1	F128/1210/2050	Mary	Doe
2	F129/1211/2050	John	Doe

The content of every table is enclosed by these two tags: **<table>** **</table>**.

Tables have rows and columns. The rows use the [<tr>](#) element ('tr' stands for 'table row') while the column is constructed using **<td>** element that stands for table data. The table above can be created using the HTML code below:

```
<table>

<tr>
<td>Sno.</td><td>Regno</td><td> First name </td><td>Surname</td>
</tr>

<tr>
<td>1</td><td> F128/1210/2050</td><td>Mary</td><td>Doe</td>
</tr>

<tr>
<td>2</td><td> F128/1211/2050</td><td>John</td><td>Doe</td>
</tr>

</table>
```

Working with Table headers

Table headers are defined with **<th>** elements. Each **<th>** element represents a header cell in an HTML table.

An HTML table has two kinds of cells:

- Header cells - contains header information (created with the **<th>** element)

- Data cells - contains data (created with the [<td>](#) element)

The text in `<th>` elements are bold and centered by default unlike the text in `<td>` elements are regular and left-aligned by default.

WORKING WITH FORMS

HTML forms are simple form that has been used to collect data from the users. HTML form has interactive controls and various input types such as text, numbers, email, password, radio buttons, checkboxes, buttons, etc. We can see its application in various areas, including registration forms, customer feedback forms, online survey forms and many more.

The HTML [`<form>`](#) tag is used to create an HTML form.

Syntax:

```
<form action="" method="">
```

```
</form>
```

The **Action** attribute represents where the data collected using the form will be submitted to

Possible values are: an absolute url example `action="http://www.seku.ac.ke/register.htm"` or a relative url `action="register.htm"` that points to a file

The **method** attribute specifies how to send form-data (the form-data is sent to the page specified in the `action` attribute).

The form-data can be sent as URL variables (with `method="get"`) or as HTTP post transaction (with `method="post"`).

The GET:

- Appends form-data into the URL in name/value pairs
- The length of a URL is limited (about 3000 characters)
- Never use GET to send sensitive data! (will be visible in the URL)
- GET is better for non-secure data, like query strings in Google

The POST:

- Appends form-data inside the body of the HTTP request (data is not shown in URL)
- Has no size limitations

- Form submissions with POST cannot be bookmarked

Example of a form

First name:

Last name:

Gender:

email:

password:

```
<html>
<head>
</head>
<body>
<form action="" method="POST">
  <label for="firstname">First name: </label>
  <input type="text" name="firstname" required>
  <br>
  <label for="lastname">Last name: </label>
  <input type="text" name="lastname" required>
  <br>
  <label for="country">Gender: </label>
  <select name="country" id="country">
    <option>Select an option</option>
    <option value="male">Male</option>
    <option value="female">Female</option>
  </select>
  <br>
  <label for="email">email: </label>
  <input type="email" name="email" required>
  <br>
  <label for="password">password: </label>
  <input type="password" name="password" required>
  <p>&nbsp;</p>
  <button type="submit">Submit</button>
  <button type="reset">Clear</button>
</form>
</body>
</html>
```

TWO WAYS WE CAN IMPROVE THE STRUCTURE/ ARRANGEMENT OF THE FORM ELEMENTS

- 1) **We can use a table:** A *table in HTML* consists of table cells inside rows and columns
- 2) **We can use CSS :** CSS is the language we use to style an HTML document. CSS describes how HTML elements should be displayed.

Using table:

```
<html>
<head></head>
<body>
<form action="register.php" method="post">
<table>
  <tr>
<td><label for="firstname">First name: </label></td>
  <td><input type="text" name="firstname" required></td>
</tr><tr>
<td><label for="lastname">Last name: </label></td>
  <td><input type="text" name="lastname" required></td>
</tr><tr>
<td><label for="country">Gender: </label></td>
  <td><select name="country" id="country">
    <option>Select an option</option>
    <option value="male">Male</option>
    <option value="female">Female</option>
  </select></td>
</tr><tr>
<td><label for="email">email: </label></td>
  <td><input type="email" name="email" required> </td>
</tr><tr>
  <td><label for="password">password: </label></td>
  <td><input type="password" name="password" required></td>
</tr><tr>
<td colspan="2" align="right"><button type="submit">Submit</button>
<button type="reset">Clear</button> </td></tr>
</table>
</form>
</body>
</html>
```

First name:

Last name:

Gender:

email:

password:

Using CSS

```
<html>
<head>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      margin: 0;
      padding: 0;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }

    form {
      background-color: #fff;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
      width: 300px;
    }

    label {
      display: block;
      margin-bottom: 8px;
      font-weight: bold;
    }

    input, select {
      width: 100%;
      padding: 8px;
```

```

        margin-bottom: 15px;
        border: 1px solid #ccc;
        border-radius: 4px;
        font-size: 16px;
    }

    button {
        width: 48%;
        padding: 10px;
        margin-top: 10px;
        border: none;
        border-radius: 4px;
        background-color: #007bff;
        color: white;
        font-size: 16px;
        cursor: pointer;
    }

    button[type="reset"] {
        background-color: #dc3545;
    }

    button:hover {
        opacity: 0.9;
    }

    .button-container {
        display: flex;
        justify-content: space-between;
    }
</style>
</head>
<body>
<form>
    <label for="firstname">First name:</label>
    <input type="text" name="firstname" required>

    <label for="lastname">Last name:</label>
    <input type="text" name="lastname" required>

    <label for="gender">Gender:</label>
    <select name="gender" id="gender">
        <option>Select an option</option>
        <option value="male">Male</option>
        <option value="female">Female</option>
    </select>

```



```
<label for="email">Email:</label>
<input type="email" name="email" required>

<label for="password">Password:</label>
<input type="password" name="password" required>

<div class="button-container">
  <button type="submit">Submit</button>
  <button type="reset">Clear</button>
</div>
</form>
</body>
</html>
```

NB: The flex layout allows responsive elements within a container to be automatically arranged depending on viewport size

A registration form with the following fields and buttons:

- First name:** A text input field.
- Last name:** A text input field.
- Gender:** A dropdown menu with the text "Select an option" and a downward arrow.
- Email:** A text input field.
- Password:** A text input field.
- Submit:** A blue button.
- Clear:** A red button.

ASSIGNMENT:

Create a book registration form for SEKU library, use CSS to make your form look more elegant.

TOPIC 3 JAVASCRIPT PART ONE

JAVASCRIPT

PART ONE

What is JavaScript?

- JavaScript is a high-level, dynamic, and interpreted programming language primarily used to make web pages interactive.
- It runs on the client-side (in browsers) and is widely used in modern web development.

JavaScript allows you to:

- **Manipulate HTML/CSS:** Change web content dynamically.
- **Handle Events:** Respond to user actions like clicks or form submissions.
- **Interact with Web APIs:** Fetch data from servers without reloading the page (e.g., using AJAX or Fetch API).
- **Build Dynamic Applications:** Create rich user interfaces and single-page applications (SPAs).

There are two main ways to include JavaScript in your HTML files

1. Inline Script (Inside HTML):

You can write JavaScript code inside the `<script>` tag within your HTML document.

2. External Script (Separate .js file):

You can write JavaScript code in a separate .js file and link it in your HTML using the `<script>` tag.

1) Inline Script (Inside HTML):

```
<html>
<head>
  <title>JavaScript Example</title>
</head>
<body>
  <h1>Welcome to JavaScript!</h1>
  <p id="demo"></p>

  <script>
    document.getElementById("demo").innerHTML = "Hello, JavaScript!";
  </script>
</body>
</html>
```

Terms to note

NB:

1) **document.getElementById()** is a method in JavaScript that retrieves an element from the HTML DOM (Document Object Model) using its unique id attribute. It is one of the most commonly used DOM methods for interacting with specific elements in the webpage

2) **Document Object Model (DOM)** is a programming interface for web documents. It represents the structure of an HTML or XML document as a tree of objects (or nodes) that can be manipulated using programming languages such as JavaScript.

2. External Script (Separate .js file):

Index.html

```
<html>
<head>
  <title>JavaScript Example</title>
  <script src="script.js"></script>
</head>
<body>
  <h1>Welcome to JavaScript!</h1>
  <p id="demo"></p>
</body>
</html>
```

2. External Script (Separate .js file): ...cont

Script.js

```
document.getElementById("demo").innerHTML = "Hello from External JavaScript!";
```

Basic JavaScript Concepts

1. Variables

Variables store data that can be used and manipulated in a program. In JavaScript, we can declare variables using `var`, `let`, or `const`.

- a) **let**: Declares a block-scoped variable.
- b) **const**: Declares a constant, block-scoped variable (its value cannot change).
- c) **var**: Declares a function-scoped variable (older and less preferred).

Basic JavaScript Concepts

1. Variables

- `let name = "John";`
- `const age = 30;`
- `var country = "Kenya";`

Basic JavaScript Concepts

2. Data Types

JavaScript has dynamic typing, meaning variables can hold any type of data. Common data types include:

- **String:** Text enclosed in quotes. Example: "Hello"
- **Number:** Numeric values. Example: 10, 3.14
- **Boolean:** True or false. Example: true, false
- **Array:** An ordered list of values. Example: [1, 2, 3]
- **Object:** A collection of key-value pairs. Example: {name: "John", age: 30}

Basic JavaScript Concepts

2. Data Types

- `let name = "Alice"; // String`
- `let age = 25; // Number`
- `let isStudent = true; // Boolean`
- `let hobbies = ["Reading", "Coding"]; // Array`
- `let person = {name: "Bob", age: 35}; // Object`

Basic JavaScript Concepts

3. Functions

A function is a block of reusable code that performs a specific task. It is declared using the function keyword.

```
function greet() {  
    console.log("Hello, world!");  
}
```

`greet();` // Calls the function and outputs: Hello, world!

Basic JavaScript Concepts

3. Functions

Functions with Parameters: You can pass values (parameters) to functions.

```
function greet(name) {  
    console.log("Hello, " + name);  
}
```

`greet("Alice");` // Outputs: Hello, Alice

Basic JavaScript Concepts

4. Conditionals (if/else)

- Conditional statements allow you to perform different actions based on certain conditions.

Basic JavaScript Concepts

4. Conditionals (if/else)

```
let score = 85;
```

```
if (score >= 90) {  
    console.log("Excellent!");  
} else if (score >= 75) {  
    console.log("Good job!");  
} else {  
    console.log("Keep improving!");  
}
```

Basic JavaScript Concepts

5. Loops

Loops allow you to execute a block of code repeatedly.

For Loop:

```
for (let i = 1; i <= 5; i++) {  
    console.log("Iteration: " + i);  
}
```

Basic JavaScript Concepts

5. Loops

While Loop:

```
let i = 1;  
while (i <= 5) {  
    console.log("Count: " + i);  
    i++;  
}
```

Basic JavaScript Concepts

5. Loops

Foreach loop

```
let fruits = ["Apple", "Banana", "Orange", "Mango", "Pineapple"];

function loopThroughArray() {
  let output = "";
  for (let i = 0; i < fruits.length; i++) {
    output += fruits[i] + "<br>"; // Append each fruit to the output
  }
}
```

Example

```
<html>
<head> <title>Array Loop Example</title></head>
<body>
<h2>Looping Through an Array in JavaScript</h2>
<button onclick="loopThroughArray()">Loop Through
Array</button>
<p id="output"></p>
<script>
  // Example array of fruits
  let fruits = ["Apple", "Banana", "Orange", "Mango", "Pineapple"];
  function loopThroughArray() {
    let output = "";
```

```
// Using a for loop to iterate through the array
```

```
for (let i = 0; i < fruits.length; i++) {
```

```
    output += fruits[i] + "<br>"; // Append each fruit to the output
```

```
}
```

```
// Displaying the result in the paragraph with id "output"
```

```
document.getElementById("output").innerHTML = output;
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Output

Looping Through an Array in JavaScript

Loop Through Array

Looping Through an Array in JavaScript

Loop Through Array

Apple
Banana
Orange
Mango
Pineapple

Assignment

- Modify the code in Example 1 to use a while loop and include a reset button that can be clicked on to clear the list of fruits.

Example 2

- Example array 2:

```
<html>
  <head> <title>Array Loop with Map Example</title></head>
<body>
  <h2>Looping Through an Array Using map()</h2>
  <!-- Button to trigger array map operation -->
  <button onclick="mapArray()">Display Array Values</button>
  <p id="arrayoutput"></p>
  <script>
    // Example array of numbers
    let numbers = [1, 2, 3, 4, 5];
    // Function to loop through the array using map and display values
```

Example 2

```
function mapArray() {  
    // Using map to simply return the same array values  
    let arrayValues = numbers.map(function(number) {  
        return number;  
    });  
  
    // Displaying the result in the paragraph with id "output"  
    document.getElementById("arrayoutput").innerHTML = "Array Values: " +  
arrayValues.join(", ");  
}  
</script>  
</body>  
</html>
```

Example 2: Output

Looping Through an Array Using map()

Display Array Values

Array Values: 1, 2, 3, 4, 5

Working with Events

- JavaScript lets you execute code when events are detected
- JavaScript can respond to user actions (events) such as clicks, keypresses, and mouse movements.

Mouse Events:

- click: Fired when an element is clicked.
- dblclick: Fired when an element is double-clicked.
- mouseover: Triggered when the mouse pointer moves over an element.
- mouseout: Fired when the mouse pointer moves out of an element.
- mousedown: Triggered when a mouse button is pressed down on an element

Working with Events

Keyboard Events:

- `keydown`: Triggered when a key is pressed down.
- `keyup`: Fired when a key is released.
- `keypress`: Triggered when a key is pressed and held (deprecated in modern browsers).

Working with Events

Form Events:

- submit: Fired when a form is submitted.
- change: Triggered when the value of an input, select, or textarea element changes.
- focus: Fired when an element gains focus.
- blur: Triggered when an element loses focus.

Working with Events

Window Events:

- **load:** Fired when the entire page (including all dependent resources like images) is fully loaded.
- **resize:** Triggered when the window is resized.
- **scroll:** Fired when the user scrolls through the page.
- **unload:** Triggered when the user leaves the page.

Working with Events

Touch Events (for mobile devices):

- `touchstart`: Fired when a touch starts.
- `touchmove`: Triggered when the touch is moved along the surface.
- `touchend`: Fired when the touch ends.

Working with Events

Handling Events in JavaScript

1) Inline Event Handling (in HTML):

You can attach event listeners directly to HTML elements using attributes like onclick, onchange, etc.

```
<button onclick="alert('Button clicked!')">Click Me</button>
```

Example 3

```
<html>
<body>
<h2>Click the Button</h2>
<button onclick="showMessage()">Click Me</button>
<script>
    function showMessage() {
        alert("Button was clicked!");
    }
</script>
</body>
</html>
```

Example 3

Click the Button

Click Me



Working with Events

Handling Events in JavaScript

2) Using Event Listeners (Preferred method):

Event listeners allow you to separate HTML structure from JavaScript code.

```
const button = document.getElementById("myButton");  
button.addEventListener("click", function() {  
    alert("Button clicked!");  
});
```

Example 4

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <title>Click Event</title>  
</head>  
<body>  
    <button id="myButton">Click Me</button>
```

```
<script>
  const button = document.getElementById("myButton");
  button.addEventListener("click", function() {
    button.textContent = "You clicked me!";
  });
</script>
</body>
</html>
```

Example 4: Output

Click Me

You clicked me!

Working with Events

3 Event Object:

When an event occurs, an event object is passed to the event handler. It contains information about the event, such as the element that triggered it, mouse coordinates, or the pressed key

```
document.addEventListener("click", function(event) {  
    console.log(event.target); // The element that was clicked  
});
```

Example 5

```
<html >
```

```
<head>
```

```
    <title>Click Event</title>
```

```
</head>
```

```
<body>
```

```
    <button id="myButton">Click Me</button>
```

```
<script>
    document.getElementById("myButton").onclick = function(event) {
        event.target.textContent = "You clicked me!";
    };
</script>
</body>
</html>
```

Assignment

- 1) Rewrite Example 3,4 and 5 codes by separating the JavaScript code in a separate .js file.
- 2) Write your own similar codes to demonstrate:
 - a) Inline Event Handling
 - b) Using Event Listeners
 - c) Event Object:

TOPIC 4 JAVASCRIPT PART TWO

JAVASCRIPT

PART TWO

Working with DOM

- The **Document Object Model (DOM)** is a programming interface for web documents.
- It represents the structure of a web page as a tree of objects, where each element (such as headings, paragraphs, or buttons) is a node.
- JavaScript can manipulate these nodes to change the content, structure, and style of the webpage dynamically.

Working with DOM

Why is the DOM Important?

By interacting with the DOM, JavaScript can:

- Modify HTML elements
- Change CSS styles
- React to user events (like clicks or keyboard inputs)
- Create, modify, or remove elements dynamically

Working with DOM

Accessing DOM Elements

- JavaScript provides several methods to select and access elements in the DOM.
- **1. getElementById()**
- 2. getElementsByClassName()
- 3. getElementsByTagName()
- 4. querySelector()
- 5. querySelectorAll()

Working with DOM

Accessing DOM Elements

- JavaScript provides several methods to select and access elements in the DOM.
- **1. getElementById()**
- This method is used to select a single HTML element by its id attribute.
- `<p id="demo">This is a paragraph.</p>`
- `<script>`
- `let element = document.getElementById("demo");`
- `console.log(element.innerHTML); // Output: This is a paragraph.`
- `</script>`
- **innerHTML** retrieves the content inside the element
-

Working with DOM

1. getElementById()

```
<html><head></head>
```

```
<body>
```

```
<p id="demo">This is a paragraph.</p>
```

```
<script>
```

```
  let element = document.getElementById("demo");
```

```
  console.log(element.innerHTML); // Output: This is a paragraph.
```

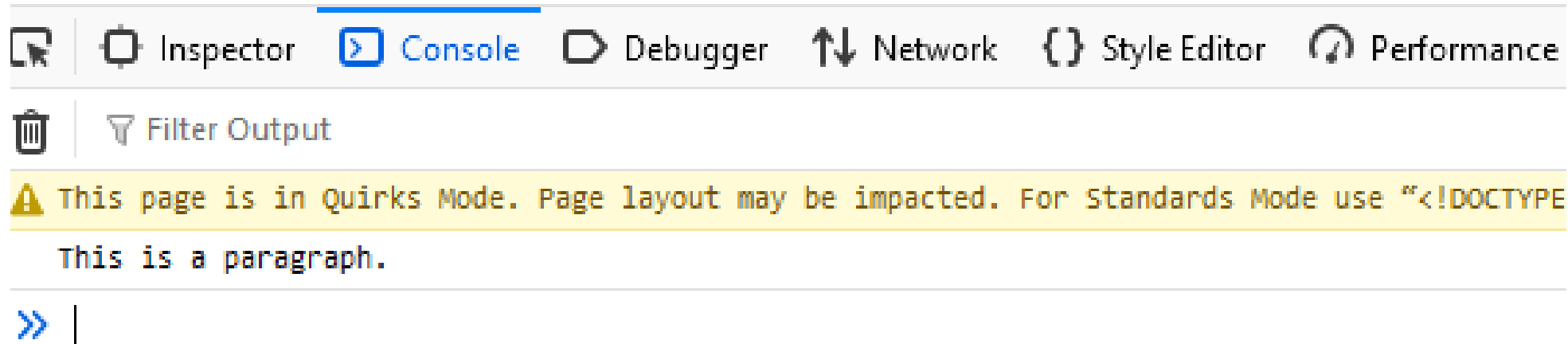
```
</script>
```

```
</body>
```

```
</html>
```

Working with DOM

1. getElementById()



Working with DOM

Accessing DOM Elements

2. `getElementsByClassName()`

- This method returns all elements with a given class name as an HTMLCollection (array-like).
- `<p class="text">First paragraph</p>`
- `<p class="text">Second paragraph</p>`
-
- `<script>`
- `let elements = document.getElementsByClassName("text");`
- `console.log(elements[0].innerHTML); // Output: First paragraph`
- `console.log(elements[1].innerHTML); // Output: Second paragraph`
- `</script>`
-

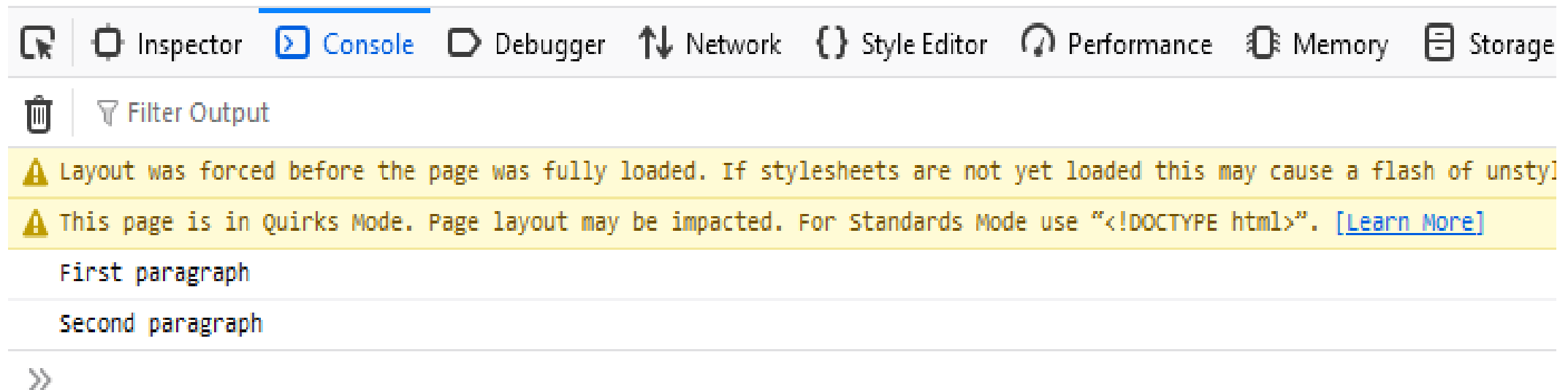
Working with DOM

2. `getElementsByClassName()`

```
<html><head></head>
<body>
<p class="text">First paragraph</p>
<p class="text">Second paragraph</p>
<script>
  let elements = document.getElementsByClassName("text");
  console.log(elements[0].innerHTML); // Output: First paragraph
  console.log(elements[1].innerHTML); // Output: Second paragraph
</script>
</body>
</html>
```

Working with DOM

2. `getElementsByClassName()`



Working with DOM

Accessing DOM Elements

3. `getElementsByName()`

This method selects elements by their tag name (e.g., `div`, `p`, `h1`).

```
<h1>This is a heading</h1>
```

```
<p>This is a paragraph.</p>
```

```
<p>This is another paragraph.</p>
```

```
<script>
```

```
    let paragraphs = document.getElementsByTagName("p");
```

```
    console.log(paragraphs[0].innerHTML); // Output: This is a paragraph.
```

```
</script>
```

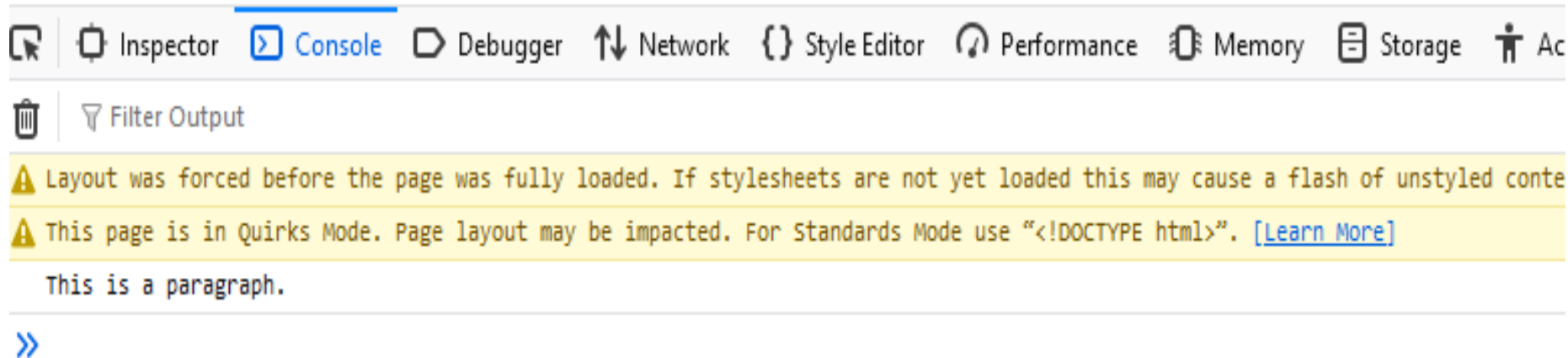
Working with DOM

3. `getElementsByTagName()`

```
<html><head></head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
<p>This is another paragraph.</p>
<script>
  let paragraphs = document.getElementsByTagName("p");
  console.log(paragraphs[0].innerHTML); // Output: This is a paragraph.
</script>
</body>
</html>
```


Working with DOM

3. `getElementsByName()`



Working with DOM

Accessing DOM Elements

- **4. `querySelector()`**
- This method returns the first element that matches a given CSS selector (like a class, id, or tag).
- `<p id="demo" class="text">Hello, world!</p>`
-
- `<script>`
- `let element = document.querySelector("#demo"); // Select by ID`
- `console.log(element.innerHTML); // Output: Hello, world!`
- `</script>`

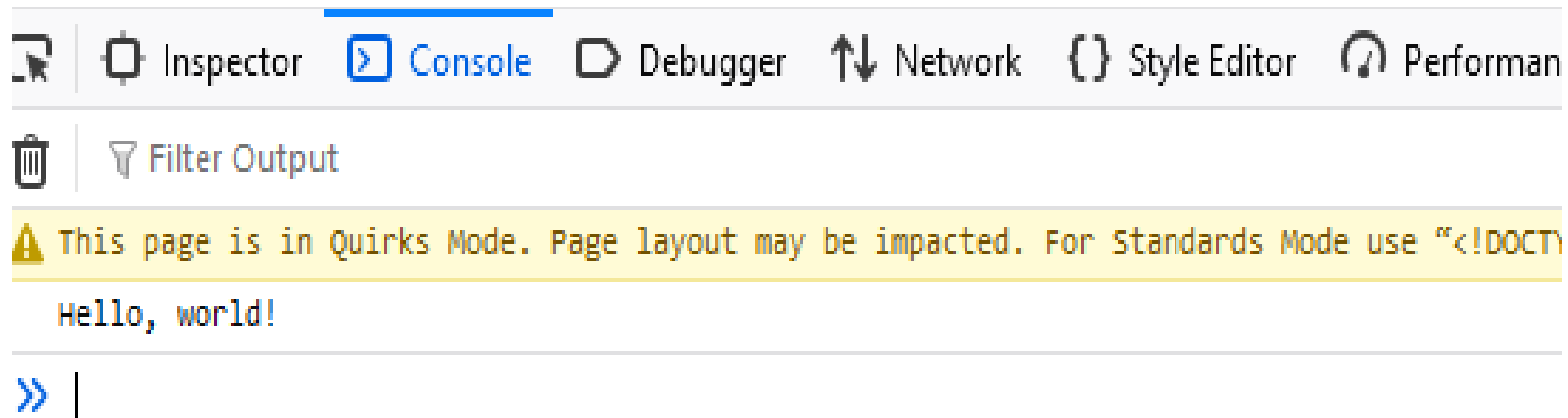
Working with DOM

4. `querySelector()`

```
<html><head></head>
<body>
<p id="demo" class="text">Hello, world!</p>
<script>
  let element = document.querySelector("#demo"); // Select by ID
  console.log(element.innerHTML); // Output: Hello, world!
</script>
</body>
</html>
```

Working with DOM

4. `querySelector()`



Working with DOM

Accessing DOM Elements

- 5. `querySelectorAll()`
- This method returns all elements that match a given CSS selector as a `NodeList`.
- `<p class="text">First paragraph</p>`
- `<p class="text">Second paragraph</p>`
-
- `<script>`
- `let elements = document.querySelectorAll(".text"); // Select by class`
- `console.log(elements[0].innerHTML); // Output: First paragraph`
- `</script>`

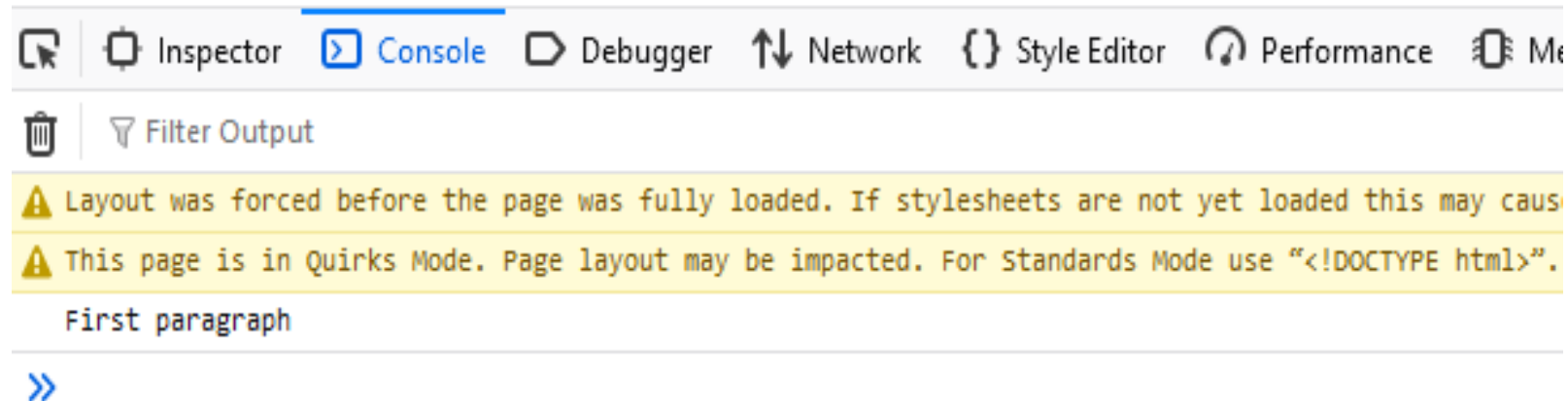
Working with DOM

5. `querySelectorAll()`

```
<html><head></head>
<body>
<p class="text">First paragraph</p>
<p class="text">Second paragraph</p>
<script>
  let elements = document.querySelectorAll(".text"); // Select by class
  console.log(elements[0].innerHTML); // Output: First paragraph
</script>
</body>
</html>
```

Working with DOM

5. `querySelectorAll()`



Modifying DOM Elements

Once you've accessed an element, you can manipulate it by changing its content, attributes, or styles.

1. Changing Content (innerHTML)

You can modify the HTML content inside an element using the innerHTML property.

2. Changing Attributes

You can use `setAttribute()` and `getAttribute()` to manipulate element attributes.

3. Changing CSS Styles

You can change the inline CSS styles of an element using the `style` property.

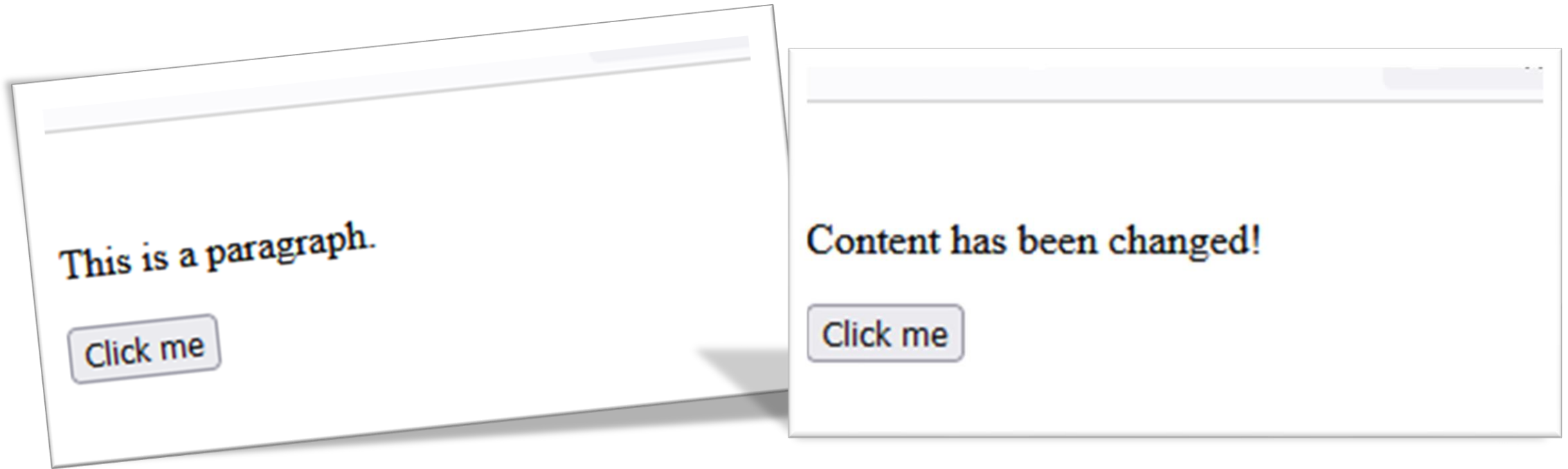
Modifying DOM Elements

1. Changing Content (innerHTML)

```
<html><head></head>
<body>
<p id="demo">This is a paragraph.</p>
<button onclick="changeContent()">Click me</button>
<script>
  function changeContent() {
    document.getElementById("demo").innerHTML = "Content has been changed!";
  }
</script>
</body>
</html>
```

Modifying DOM Elements

1. Changing Content (innerHTML)



Modifying DOM Elements

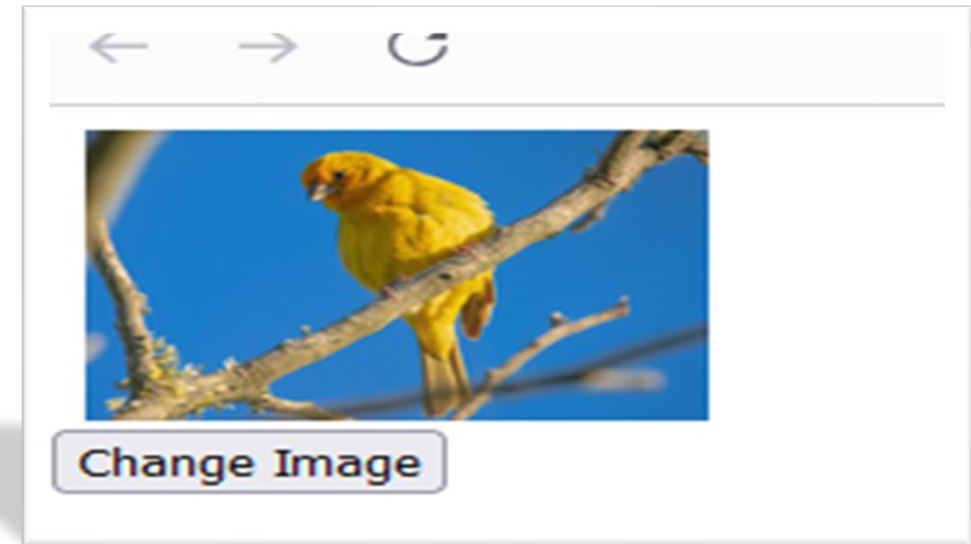
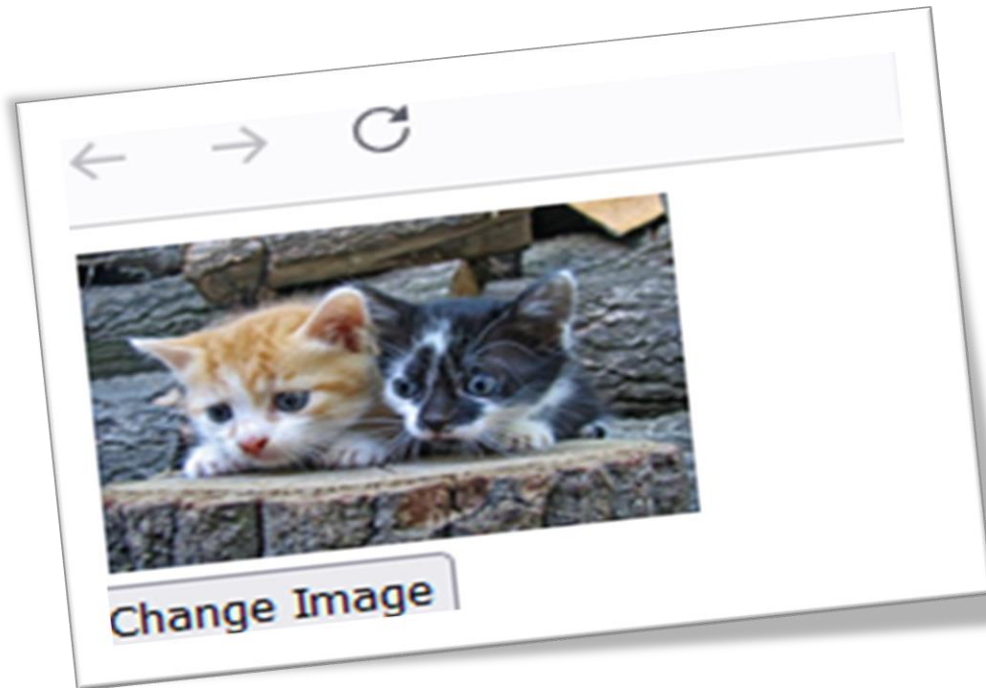
2. Changing Attributes

```
<html><head></head>
<body>

<br>
<button onclick="changeImage()">Change Image</button>
<script>
  function changeImage() {
    let img = document.getElementById("image");
    img.setAttribute("src","birds.jpg");
    img.setAttribute("width", "150");
  }
</script>
</body>
</html>
```

Modifying DOM Elements

2. Changing Attributes



Modifying DOM Elements

3. Changing CSS Styles

```
<html><head></head>

<body>

<p id="text">Change my color!</p>

<button onclick="changeColor()">Change Color</button>

<script>

    function changeColor() {

        document.getElementById("text").style.color = "blue";

    }

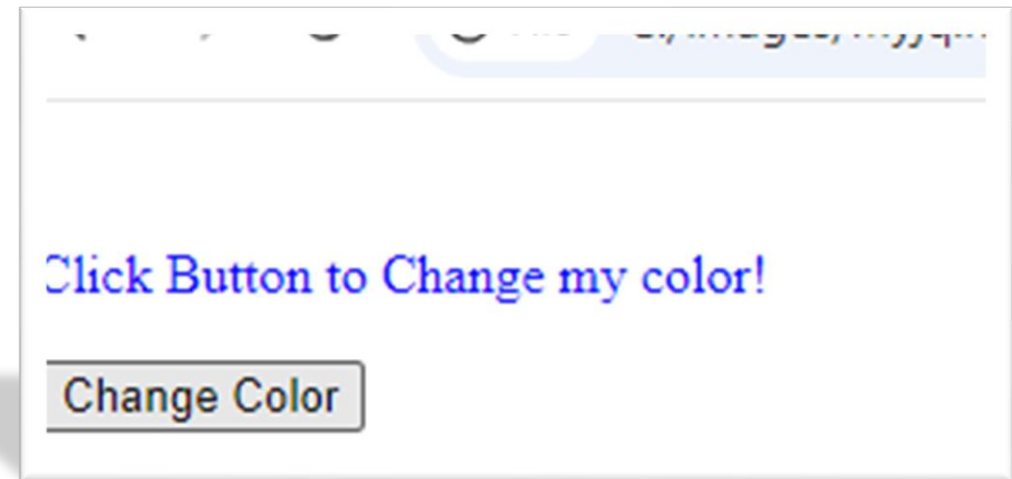
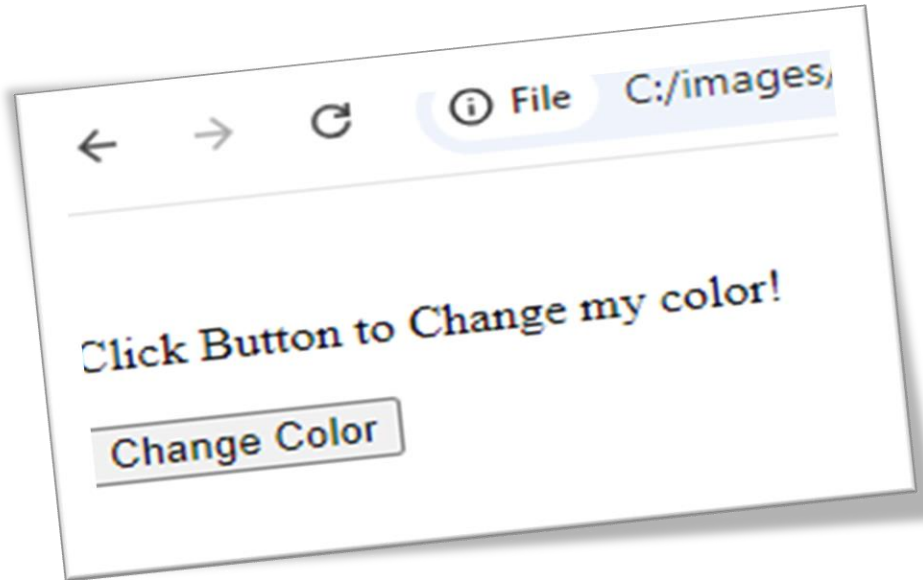
</script>

</body>

</html>
```

Modifying DOM Elements

3. Changing CSS Styles



Adding and Removing Elements

You can dynamically create, append, or remove elements from the DOM using JavaScript.

1. Creating and Appending Elements

- Use `createElement()` to create a new element, and `appendChild()` to add it to the DOM.

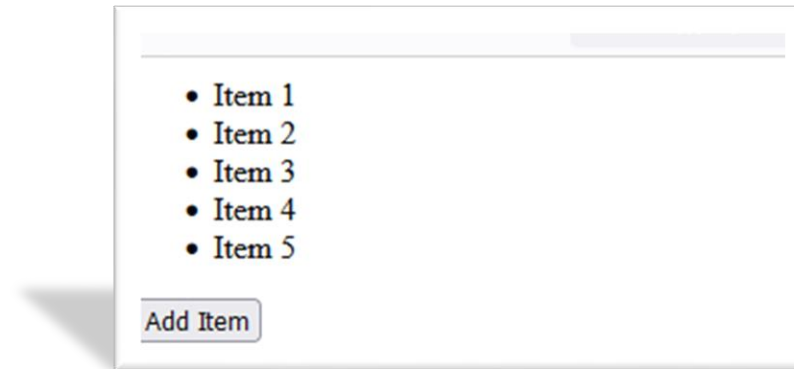
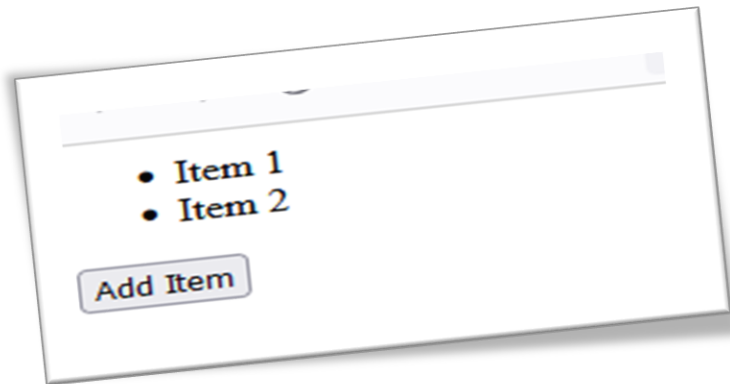
Adding and Removing Elements

1. Creating and Appending Elements

```
<html><head></head>
<body>
  <ul id="list">
    <li>Item 1</li>
    <li>Item 2</li>
  </ul>
  <button onclick="addItem()">Add Item</button>
  <script>
    let itemCount = 3;
    function addItem() {
      let newItem = document.createElement("li");
      newItem.innerHTML = "Item " + itemCount;
      document.getElementById("list").appendChild(newItem);
      itemCount++;
    }
  </script>
</body>
</html>
```


Adding and Removing Elements

- You can dynamically create, append, or remove elements from the DOM using JavaScript.
- **1. Creating and Appending Elements**



Adding and Removing Elements

- **2. Removing Elements**
- You can remove an element using `removeChild()` or the `remove()` method.

Adding and Removing Elements

2. Removing Elements

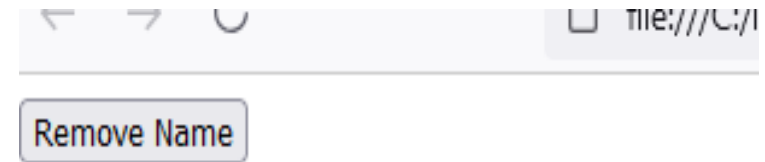
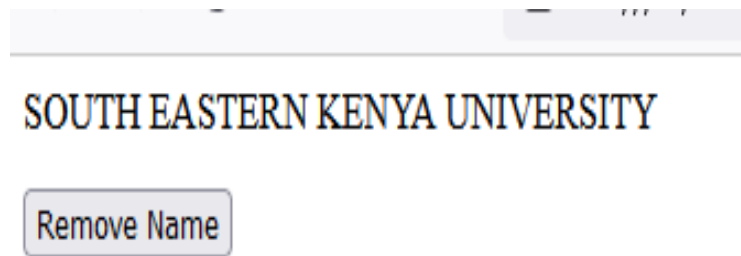
```
<html><head></head>
<body>
<p id="demo">SOUTH EASTERN KENYA UNIVERSITY</p>
<button onclick="removeElement()">Remove Name</button>
<script>
  function removeElement() {
    let element = document.getElementById("demo");
    element.remove(); // Removes the element
  }
</script>
</body>
</html>
```

.

Adding and Removing Elements

2. Removing Elements

.



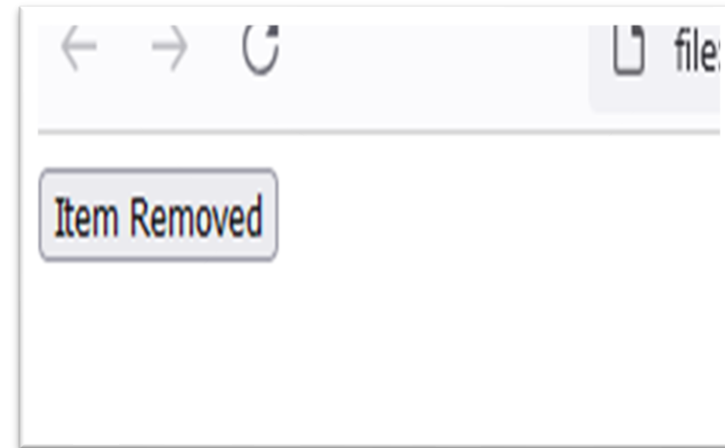
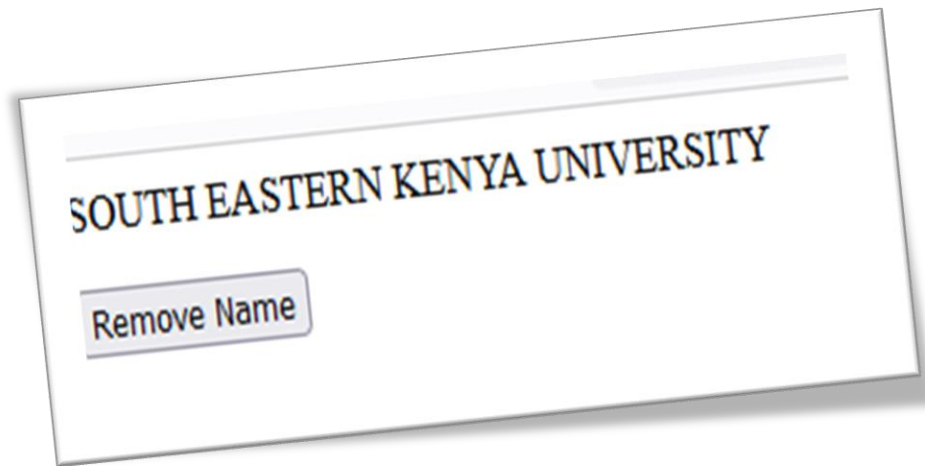
Removing and Modify Elements

2. Removing Elements

```
<html><head></head>
<body>
<p id="demo">SOUTH EASTERN KENYA UNIVERSITY</p>
<button id="btn" onclick="removeElement()">Remove Name</button>
<script>
  function removeElement() {
    let element = document.getElementById("demo");
    element.remove(); // Removes the element
    document.getElementById("btn").innerHTML = "Name Removed";
  }
</script>
</body>
</html>
```

Removing and Modify Elements

2. Removing Elements



Handling Events

The DOM allows you to handle various events like clicks, key presses, and more. You can use event listeners to respond to user actions.

1. Handling Click Events

- You can use the onclick attribute to define actions on a button or use `addEventListener()` for more flexibility.

```
<html><head></head>
<body>
<button onclick="handleClick()">Click me</button>
<script>
  function handleClick() {
    alert("Button clicked!");
  }
</script>
</body>
</html>
```

Handling Events



Handling Events

2. Using `addEventListener()`

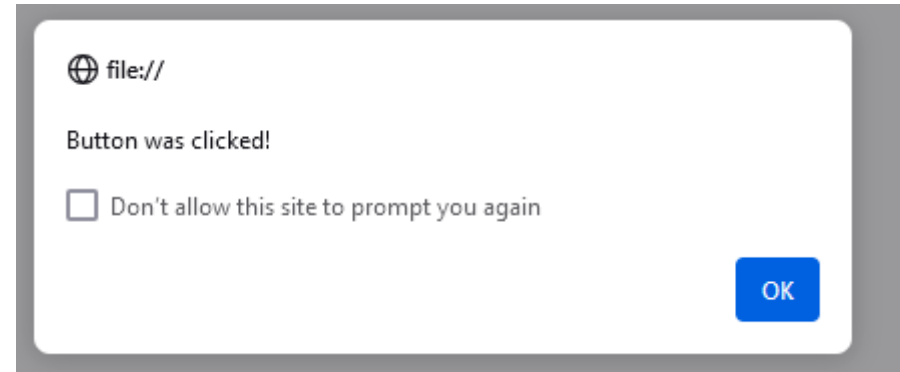
This method allows you to attach multiple events to an element without overwriting existing events.

```
<button id="myButton">Click me</button>

<script>
    document.getElementById("myButton").addEventListener("click", function() {
        alert("Button was clicked!");
    });
</script>
```

Handling Events

2. Using `addEventListener()`



Handling Events

3. Handling Form Input Events

You can listen to events like input and submit for form interactions.

```
<input type="text" id="nameInput" placeholder="Type your name">
<p id="output"></p>
<script>
  document.getElementById("nameInput").addEventListener("input", function() {
    let name = this.value;
    document.getElementById("output").innerHTML = "Hello, " + name;
  });
</script>
```

Handling Events

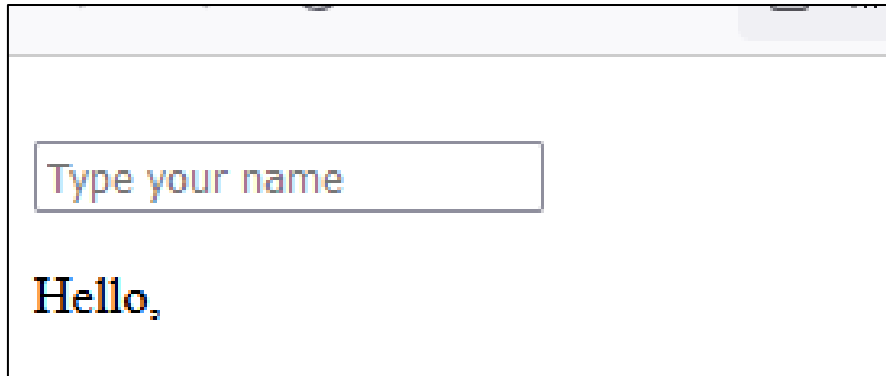
3. Handling Form Input Events

You can listen to events like input and submit for form interactions.

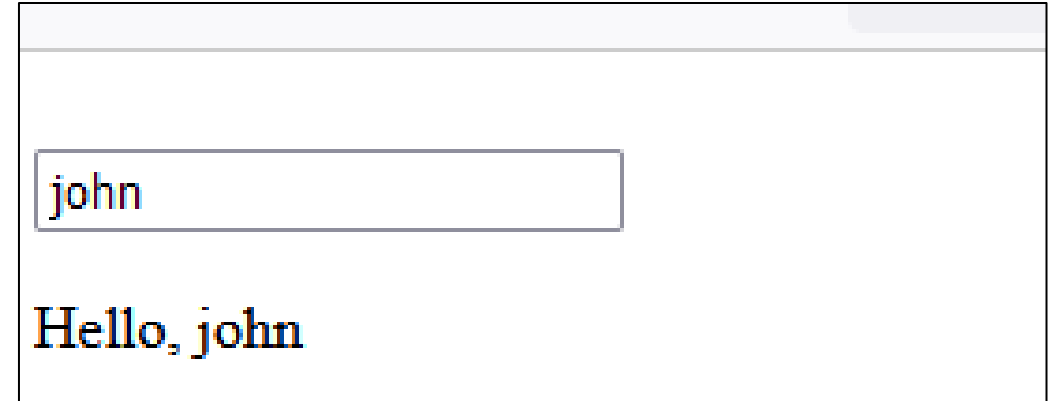
```
<html><head></head>
<body>
<form action="" method="">
<input type="text" id="nameInput" placeholder="Type your name"><br>
<p id="output"></p>
<script>
  document.getElementById("nameInput").addEventListener("input", function() {
    let name = this.value;
    document.getElementById("output").innerHTML = "Hello, " + name;
  });
</script>
</form>
</html>
```

Handling Events

3. Handling Form Input Events



A browser window with a light gray title bar. Inside, there is a text input field with the placeholder text "Type your name". Below the input field, the text "Hello," is displayed.



A browser window with a light gray title bar. Inside, the text input field now contains the text "john". Below the input field, the text "Hello, john" is displayed.

Assignment

- Create a calculator application using HTML, CSS and Javascript. In your application you must demonstrate the use of DOM and functions..

Working with functions

Java script function you start with keyword function then name of the function

//standard function

```
function Marks(m){  
    return m+20;  
}  
Marks(50)
```

// arrow function

```
const Studentmarks=(m)=>{  
    return m+20;  
}
```

//short form of arrow function

```
const Mystudentmarks=m=>m+20;
```

Working with functions: Objects inside Javascript

```
let myObj={  
  fname: "John",  
  lname: "Doe",  
  age:28,  
  gender: "male",  
  courseunits:["WebProg","DBSystems","Algebra1"]  
}  
console.log(myObj.fname)  
console.log(myObj["lname"])  
console.log(myObj.age)  
console.log(myObj.courseunits)
```


Working with functions: Objects inside Javascript

```
JS main.js
JS main.js > [x] myObj
1  let myObj={
2      fname: "John",
3      lname: "Doe",
4      age:28,
5      gender: "male",
6
7      courseunits:["WebProg","DBSystems","Algebra1"]
8  }
9      console.log(myObj.fname)
10     console.log(myObj["lname"])
11     console.log(myObj.age)
12     console.log(myObj.courseunits)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\pkasy\Desktop\JAVASCRIPT> node main.js
John
Doe
PS C:\Users\pkasy\Desktop\JAVASCRIPT>
PS C:\Users\pkasy\Desktop\JAVASCRIPT> node main.js
John
Doe
28
[ 'WebProg', 'DBSystems', 'Algebra1' ]
PS C:\Users\pkasy\Desktop\JAVASCRIPT>
```

Working with functions: Objects inside Javascript

```
let myObj={
  fname: "John",
  lname: "Doe",
  age:28,
  gender: "male",
  courseunits:["WebProg","DBSystems","Algebra1"],
  parent: {
    name: "Jane Doe",
    telno: 072222222,
    address: "Nairobi Kenya"
  }
}
console.log(myObj.fname)
console.log(myObj["lname"])
console.log(myObj.age)
console.log(myObj.courseunits)
console.log(myObj.parent.name)
console.log(myObj.parent.address)
```

Working with functions: Objects inside Javascript

```
JS main.js > ...
1  let myObj={
2      fname: "John",
3      lname: "Doe",
4      age:28,
5      gender: "male",
6      courseunits:["WebProg","DBSystems","Algebra1"],
7      parent: {
8          name: "Jane Doe",
9          telno: 0722222222,
10         address: "Nairobi Kenya"
11     }
12 }
13 console.log(myObj.fname)
14 console.log(myObj["lname"])
15 console.log(myObj.age)
16 console.log(myObj.courseunits)
17 console.log(myObj.parent.name)
18 console.log(myObj.parent.address)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pkasy\Desktop\JAVASCRIPT> node main.js
John
Doe
28
[ 'WebProg', 'DBSystems', 'Algebra1' ]
Jane Doe
Nairobi Kenya
PS C:\Users\pkasy\Desktop\JAVASCRIPT>
```

Example

```
<html>
```

```
<head></head>
```

```
<body>
```

```
  <h2>Student Information</h2>
```

```
  <div id="studentInfo"></div>
```

```
  <script>
```

```
    let myObj = {
```

```
      fname: "John",
```

```
      lname: "Doe",
```

```
      age: 28,
```

```
      gender: "male",
```

```
      courseunits: ["WebProg", "DBSystems", "Algebra1"]
```

```
    };
```

Example

```
let studentInfoDiv = document.getElementById("studentInfo");
studentInfoDiv.innerHTML = `
  <p><strong>First Name:</strong> ${myObj.fname}</p>
  <p><strong>Last Name:</strong> ${myObj.lname}</p>
  <p><strong>Age:</strong> ${myObj.age}</p>
  <p><strong>Gender:</strong> ${myObj.gender}</p>
  <p><strong>Course Units:</strong></p>
  <ul>
    ${myObj.courseunits.map(unit => `<li>${unit}</li>`).join("")}
  </ul>
`;
</script>
</body>
</html>
```

Output

```
1 // Student.js
2
3 // Student constructor function
4 function Student(firstName, lastName, age, gender, courseUnits) {
5     this.firstName = firstName;
6     this.lastName = lastName;
7     this.age = age;
8     this.gender = gender;
9     this.courseUnits = courseUnits;
10 }
11
12 // Create a new Student object
13 var student = new Student('John', 'Doe', 28, 'male', ['WebProg', 'DBSystems', 'Algebra1']);
14
15 // Print the student object
16 console.log(student);
```

Student Information

First Name: John

Last Name: Doe

Age: 28

Gender: male

Course Units:

- WebProg
- DBSystems
- Algebra1

Photo Gallery using concepts

```
<html>
<head> <title>Photo Gallery</title>
  <style>
    img {
      width: 400px;
      height: 300px;
    }
  </style>
</head>
<body>
  <h2>Photo Gallery</h2>
  <div> </div>
  <button id="backBtn">Back</button>
  <button id="nextBtn">Next</button>
```

Photo Gallery using concepts

```
<script>
  const images = [
    "animals.jpg",
    "birds.jpg",
  ];
  let currentIndex = 0;
  // Function to update the image source
  function updateImage(index) {
    let galleryImage = document.getElementById("galleryImage");
    galleryImage.setAttribute("src", images[index]);
  }
```


Photo Gallery using concepts

// Next button functionality

```
document.getElementById("nextBtn").addEventListener("click", function() {  
    currentIndex = (currentIndex + 1) % images.length; // Loop back to first image  
    updateImage(currentIndex);  
});
```

// Back button functionality

```
document.getElementById("backBtn").addEventListener("click", function() {  
    currentIndex = (currentIndex - 1 + images.length) % images.length; // Loop to last image  
    updateImage(currentIndex);  
});
```

```
</script>
```

```
</body>
```

```
</html>
```

Output

Photo Gallery



[Back](#) [Next](#)

Photo Gallery



[Back](#) [Next](#)

TOPIC 5 JQUERY, JSON, AJAX & BOOTSTRAP

INTRODUCTION TO JQUERY

1. What is jQuery?

jQuery is a lightweight JavaScript library that simplifies HTML document manipulation, event handling, animation, and AJAX interactions. It helps you write less code to achieve the same functionality you would with pure JavaScript.

2. Setting Up jQuery

To use jQuery, you need to include the jQuery library in your HTML file. You can do this by adding a `<script>` tag in the `<head>` or at the end of the `<body>` tag.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>jQuery Example</title>
  <!-- Include jQuery from a CDN -->
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>

  <!-- Your content goes here -->
  <h1 id="heading">Hello, World!</h1>
  <button id="btn">Click Me!</button>

  <script>
    // jQuery code will go here
  </script>
</body>
</html>
```

3. Basic jQuery Syntax

The basic syntax of jQuery is:

`$(selector).action();`

1. **\$**: This is the jQuery function.
2. **selector**: The element(s) you want to select.
3. **action**: The action you want to perform on the selected element(s).

4. Simple Example

EXAMPLE 1: TEXT CHANGE

Let's create a basic example where clicking a button changes the text of a heading.

```
<html>
<head>
  <title>jQuery Example</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>

  <h1 id="heading">Hello, World!</h1>
  <button id="btn">Click Me!</button>

  <script>
    // jQuery code to change the text when the button is clicked
    $(document).ready(function() {
      $("#btn").click(function() {
        $("#heading").text("Hello, jQuery!");
      });
    });
  </script>
</body>
</html>
```

Hello, SEKU

Click Me!

You click the button you get the output below

Hello, SOUTH EASTERN KENYA UNIVERSITY!

Click Me!

Explanation

- `$(document).ready(function() { ... });`: This ensures that the jQuery code runs only after the DOM is fully loaded.

- `$("#btn").click(function() { ... });`: This binds a click event to the button with the id="btn".
- `$("#heading").text("Hello, jQuery!");`: This changes the text of the element with the id="heading" to "Hello, jQuery!" when the button is clicked.

EXAMPLE 2: PARAGRAPH TOGGLE

```
<html>
<head>
  <title>jQuery Toggle Example</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>

  <button id="toggle-btn">Toggle Paragraph</button>
  <p id="paragraph">This is a paragraph that can be toggled.</p>

  <script>
    $(document).ready(function() {
      $("#toggle-btn").click(function() {
        $("#paragraph").toggle();
      });
    });
  </script>
</body>
</html>
```

Toggle Paragraph

This is a paragraph that can be toggled.

Toggle Paragraph

INTRODUCTION TO JSON

JSON (JavaScript Object Notation) is a lightweight data-interchange format that's easy for humans to read and write and easy for machines to parse and generate. It's widely used in web applications to transmit data between a server and a client.

1. What is JSON?

JSON is a text format that represents structured data based on key-value pairs. It is language-independent and is often used in APIs, configuration files, and data storage.

2. Basic JSON Syntax

- **Data is in key/value pairs:** "key": "value"
- **Data is separated by commas:** "key1": "value1", "key2": "value2"
- **Curly braces {} hold objects:** {"key": "value"}
- **Square brackets [] hold arrays:** {"key": ["value1", "value2"]}

Example of JSON data

```
{
  "name": "John Doe",
  "age": 30,
  "isStudent": false,
  "courses": ["Math", "Science", "History"],
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "zipcode": "10001"
  }
}
```

Parsing JSON

When you receive JSON data (e.g., from an API), you typically need to parse it into a usable format in your programming language.

Example

```
// JSON string
var jsonString = '{"name": "John Doe", "age": 30, "isStudent": false}';
// Parsing JSON string to JavaScript object
var jsonObject = JSON.parse(jsonString);

console.log(jsonObject.name); // Outputs: John Doe
console.log(jsonObject.age); // Outputs: 30
```

Creating JSON

You can also create JSON strings from JavaScript objects.

```
// JavaScript object
var person = {
  name: "Jane Doe",
  age: 30
};
```

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>Simple JSON Example</title>
</head>
<body>
  <h1>JSON Example</h1>
  <button id="show-data-btn">Show Data</button>
  <div id="data-section">
    <p>Click the button to display data.</p>
  </div>

  <script>
    // JSON Data
    const jsonData = {
      "name": "John Doe",
      "age": 30
    };

    // Event listener for the button
    document.getElementById('show-data-btn').addEventListener('click', () => {
      const dataSection = document.getElementById('data-section');
      dataSection.innerHTML = ""; // Clear previous content

      // Populate data dynamically
      const name = document.createElement('p');
      name.textContent = `Name: ${jsonData.name}`;

      const age = document.createElement('p');
      age.textContent = `Age: ${jsonData.age}`;

      // Append elements to the section
      dataSection.appendChild(name);
      dataSection.appendChild(age);
    });
  </script>
</body>
</html>
```

INTRODUCTION TO AJAX

AJAX (Asynchronous JavaScript and XML) is a technique used in web development to create dynamic and interactive web applications. It allows web pages to be updated asynchronously by exchanging small amounts of data with the server behind the scenes. This means that parts of a web page can be updated without having to reload the entire page.

1. What is AJAX?

- **Asynchronous:** AJAX allows web pages to send and receive data from a server asynchronously, without interfering with the display and behavior of the existing page.
- **JavaScript:** AJAX uses JavaScript to perform the asynchronous requests to the server.
- **XML (or JSON):** Originally, data was sent back in XML format, but today, JSON is more commonly used due to its simplicity and ease of use.

2. How AJAX Works

The basic flow of an AJAX request is as follows:

1. **Client Event:** A user action triggers an event on the web page (e.g., clicking a button).
2. **AJAX Request:** JavaScript creates an XMLHttpRequest object and sends a request to the server.
3. **Server Response:** The server processes the request and sends back a response, usually in JSON or XML format.
4. **Update Web Page:** JavaScript processes the server's response and updates the web page accordingly without reloading it.

3. Basic Example: Fetching Data

```
<html>
<head> <title>Simple AJAX Example</title></head>
<body>
  <h1>AJAX Example</h1>
  <button id="load-text-btn">Load Text</button>
  <div id="data-section">
    <p>Click the button to load data.</p>
  </div>

  <script>
    // Event listener for the button
    document.getElementById('load-text-btn').addEventListener('click', () => {
      const dataSection = document.getElementById('data-section');
      dataSection.innerHTML = '<p>Loading...</p>'; // Temporary loading message

      // Create an XMLHttpRequest object
      const xhr = new XMLHttpRequest();

      // Configure the request (GET request, simulated plain text response)
      xhr.open('GET', 'https://baconipsum.com/api/?type=meat-and-filler&sentences=1', true);
      //xhr.open('GET', 'example.txt', true);
      // Define what to do when the response is ready
      xhr.onload = function () {
        if (xhr.status === 200) {
```



```

        // Display the plain text response
        dataSection.innerHTML = `<p>${xhr.responseText}</p>`;
    } else {
        // Handle errors
        dataSection.innerHTML = `<p>Error: Unable to load data (Status ${xhr.status})</p>`;
    }
};

// Handle network errors
xhr.onerror = function () {
    dataSection.innerHTML = `<p>Error: Network error occurred.</p>`;
};

// Send the request
xhr.send();
});
</script>
</body>
</html>

```

DIFFERENCES BETWEEN AJAX, JSON AND JQUERY

1. jQuery

- **Type:** JavaScript library.
- **Purpose:** Simplifies tasks like DOM manipulation, event handling, animation, and AJAX.
- **Usage:** Provides methods like \$.ajax(), \$.get(), and \$.post() to make AJAX calls easier.

2. JSON (JavaScript Object Notation)

- **Type:** Data format.
- **Purpose:** Used to represent structured data in a readable and lightweight format, often for data exchange between a server and a web application.
- **Usage:** Commonly used in AJAX requests to send and receive data in a consistent format.

3. AJAX (Asynchronous JavaScript and XML)

- **Type:** Technique.
- **Purpose:** Allows asynchronous data retrieval from a server without refreshing the entire web page.
- **Usage:** Often used with JSON to fetch or send data, making web applications more dynamic and responsive.

BOOTSTRAP

Bootstrap is a popular open-source front-end framework for developing responsive and mobile-first websites. It provides a collection of CSS and JavaScript components that simplify the process of creating modern web interfaces. Bootstrap is designed to be easy to use, and it enables developers to quickly build layouts, forms, buttons, navigation, and other UI elements.

Key Features:

- **Responsive Design:** Bootstrap is built on a responsive grid system, ensuring that your web pages look great on all devices, from mobile phones to desktops.
- **Pre-built Components:** Bootstrap comes with a wide range of ready-to-use components like buttons, navigation bars, modals, carousels, forms, and more.
- **Customization:** Bootstrap is highly customizable. You can easily modify its components to suit your design needs.
- **Cross-browser Compatibility:** Bootstrap is designed to work consistently across major web browsers.

Bootstrap Grid System

The **Bootstrap Grid System** is the foundation of Bootstrap's responsive design. It uses a series of containers, rows, and columns to layout and align content. The grid system is based on a 12-column layout, which can be customized for different screen sizes and devices.

Key Concepts:

1. **Containers:**

- Containers are used to hold the grid system content. There are two types:
 - `.container`: Provides a responsive fixed-width container.
 - `.container-fluid`: Provides a full-width container, spanning the entire width of the viewport.

```
<div class="container">
  <!-- Grid content goes here -->
</div>
```

```
<div class="container-fluid">
  <!-- Grid content goes here -->
</div>
```

TOPIC 6 INTRODUCTION TO PHP

INTRODUCTION TO PHP

What is PHP?

PHP (Hypertext Preprocessor) is a server-side scripting language used to create dynamic web pages. PHP code is executed on the server, and the result is returned to the browser as plain HTML.

How to run PHP code:

1. Install XAMPP (or any local server like WAMP).
2. Save your PHP files with a .php extension.
3. Place them in the htdocs folder in XAMPP.
4. Run the XAMPP server and access the file through `http://localhost/yourfile.php`.

Basic PHP Syntax:

PHP code is placed between `<?php` and `?>` tags.

```
<?php
```

```
    echo "Hello, World!";
```

```
?>
```

Example 1: Basic PHP Program

```
<?php
```

```
    echo "Hello, this is my first PHP script!";
```

```
?>
```

Chapter 2: Variables and Data Types

Variables in PHP:

- Variables store data.
- Variables in PHP start with \$.

Rules for Variables:

- Must start with a \$ followed by a letter or underscore.
- Cannot start with a number.
- Are case-sensitive.

Example:

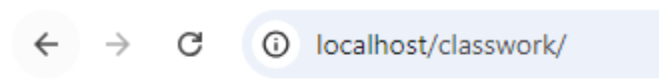
```
<?php
```

```
$name = "John";
```

```
$age = 25;
```

```
echo "Name: $name, Age: $age";
```

```
?>
```



Name: John, Age: 25

Data Types in PHP:

- **String:** Text. Example: "Hello"
- **Integer:** Whole numbers. Example: 10
- **Float:** Decimal numbers. Example: 10.5
- **Boolean:** True or false. Example: true

Example 2: Data Types in PHP

```
<?php
```

```
$name = "Alice";    // String
```

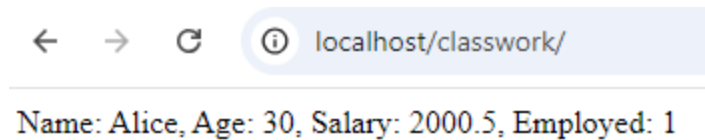
```
$age = 30;          // Integer
```

```
$salary = 2000.50;  // Float
```

```
$isEmployed = true; // Boolean
```

```
echo "Name: $name, Age: $age, Salary: $salary, Employed: $isEmployed";
```

```
?>
```



← → ↻ ⓘ localhost/classwork/

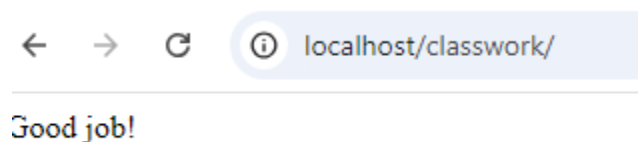
Name: Alice, Age: 30, Salary: 2000.5, Employed: 1

Chapter 3: Control Structures (If, Else, and Loops)

If-Else Statement:

Control the flow of a program by executing certain code if a condition is true.

```
<?php
$score = 85;
if ($score >= 90) {
    echo "Excellent!";
} elseif ($score >= 75) {
    echo "Good job!";
} else {
    echo "Keep improving!";
}
?>
```



← → ↻ ⓘ localhost/classwork/

Good job!

Loops in PHP:

- **While Loop:** Repeats code while a condition is true.

```
<?php
```

```
$i = 1;

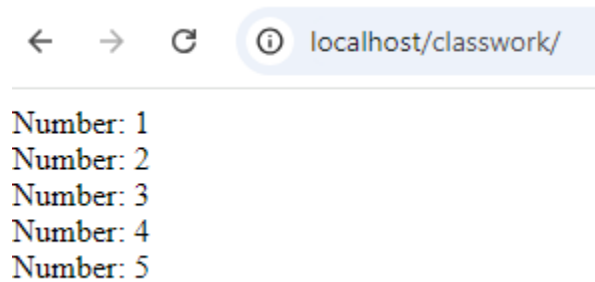
while ($i <= 5) {

    echo "Number: $i<br>";

    $i++;

}

?>
```



For Loop: Repeats code a specific number of times.

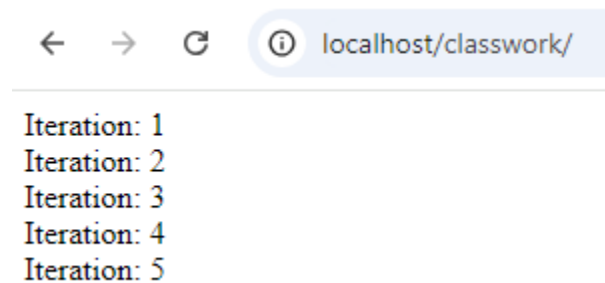
```
<?php

for ($i = 1; $i <= 5; $i++) {

    echo "Iteration: $i<br>";

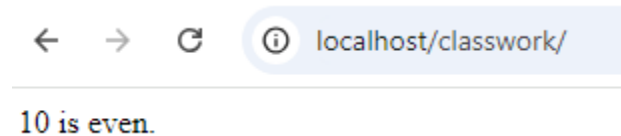
}

?>
```



Example 3: Using If-Else and Loops

```
<?php
$num = 10;
if ($num % 2 == 0) {
    echo "$num is even.<br>";
} else {
    echo "$num is odd.<br>";
}
?>
```



```
<?php

// Loop from 1 to 5

for ($i = 1; $i <= 5; $i++) {

    echo "Count: $i<br>";

}

?>
```

Functions in PHP

PHP functions are blocks of code that can be reused multiple times in a script. They allow you to organize your code into logical chunks, making it easier to read and maintain.

1. Creating a PHP Function

To define a function in PHP, you use the function keyword followed by the function name and a pair of parentheses. Here's the syntax:

```
function functionName() {

    // code to execute

}
```

Example :

```
<?php
```

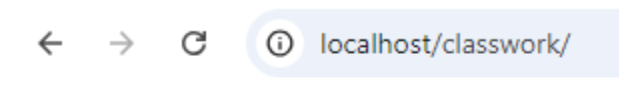
```
function greet() {
```

```
    echo "Hello, welcome to PHP functions!";
```

```
}
```

```
greet(); // Calling the function
```

```
?>
```



Hello, welcome to PHP functions!

```
<html lang="en">
```

```
<head>
```

```
    <title>Fruit List</title>
```

```
</head>
```

```
<body>
```

```
    <p>
```

```
        <?php
```

```
        function displayFruits() {
```

```
            $fruits = array("Apple", "Mango", "Banana", "Pineapple");
```

```
            $fruitCount = count($fruits);
```

```
            for ($i = 0; $i < $fruitCount; $i++) {
```

```
                echo $fruits[$i].'<br>';
```

```
            }
```

```
        }
```

```
        displayFruits();
```

```
    ?>
```

```
</p>
```

```
</body>
```

```
</html>
```

Output

Apple
Mango
Banana
Pineapple

PHP Functions with Parameters

You can pass values, called **parameters**, to a function. The parameters are specified inside the parentheses.

Example

```
function greetPerson($name) {  
    echo "Hello, $name!";  
}
```

```
greetPerson("John"); // Calling the function with an argument
```

Convert the previous code to parameter function

```
<html>  
<head>  
    <title>Fruit List</title>  
</head>  
<body>  
    <p>  
        <?php  
            function displayFruits($fruits) {
```

```

        $fruitCount = count($fruits);
        for ($i = 0; $i < $fruitCount; $i++) {
            echo $fruits[$i] . '<br>'; // Display each fruit on a new line
        }
    }
    $fruitsArray = array("Apple", "Mango", "Banana", "Pineapple");
    displayFruits($fruitsArray);
?>
</p>
</body>
</html>

```

PHP Functions with Return Values

A function can return a value using the return statement. This allows you to store or use the result elsewhere in your code.

Example:

```

function addNumbers($a, $b) {

    return $a + $b;

}

```

```
$result = addNumbers(5, 3); // Storing the result of the function
```

```
echo $result; // Outputs 8
```

Default Parameter Values

You can assign default values to function parameters. This way, if no value is provided when calling the function, the default will be used.

```

function greetPerson($name = "Guest") {

    echo "Hello, $name!";
}

```

```
}
```

```
greetPerson(); // Uses default value "Guest"
```

```
greetPerson("Alice"); // Overrides default with "Alice"
```

PHP Functions with Multiple Parameters

You can define functions with multiple parameters, separated by commas.

Example:

```
function multiply($a, $b) {
```

```
    return $a * $b;
```

```
}
```

```
echo multiply(4, 5); // Outputs 20
```

Arrow Functions in PHP

Arrow functions, introduced in PHP 7.4, are a shorthand syntax for creating anonymous functions (or closures). They provide a more concise way to write simple functions.

1. Basic Syntax of Arrow Functions

Here's the basic syntax of an arrow function:

```
fn(parameter_list) => expression;
```

Example:

```
<?php
```

```
$add = fn($a, $b) => $a + $b;
```

```
echo $add(3, 5); // Outputs 8
```

```
?>
```

- `fn` is used to declare the arrow function.
- Parameters are passed inside parentheses (`$a`, `$b`).
- The `=>` symbol separates the parameters from the expression (what the function will return).

2. Arrow Functions vs. Regular Anonymous Functions

Arrow functions are more concise but also have some differences from regular anonymous functions:

- **Single Expression:** Arrow functions can only contain a single expression. The result of this expression is automatically returned.
- **Variable Scope:** Arrow functions inherit variables from the parent scope automatically (closures), so there is no need to use the `use` keyword.

Example: Arrow Function Using Parent Scope Variables

```
<?php
```

```
$x = 10;
```

```
$multiply = fn($y) => $x * $y;
```

```
echo $multiply(5); // Outputs 50
```

```
?>
```

Here, the arrow function inherits `$x` from the parent scope without the need for the `use` keyword, which is necessary in regular anonymous functions.

3. Using Arrow Functions in Arrays

Arrow functions are handy when working with array functions like `array_map()` or `array_filter()`.

```
<?php
```

```
$numbers = [1, 2, 3, 4, 5];
```

```
// Multiply each element by 2 using array_map and an arrow function
```

```
$doubled = array_map(fn($n) => $n * 2, $numbers);
```

```
print_r($doubled); // Outputs [2, 4, 6, 8, 10]
?>
```

4. Arrow Functions with No Parameters

You can create an arrow function without parameters as well.

```
<?php
$sayHello = fn() => "Hello, world!";
echo $sayHello(); // Outputs "Hello, world!"
?>
```

OTHERS:

Arrow Functions with Closures and Multiple Scopes

Arrow functions can capture variables from the parent scope automatically without the need for the use keyword. This is useful when working with closures in nested functions.

```
<?php
function createMultiplier($factor) {
    return fn($n) => $n * $factor; // Arrow function inherits $factor
}
```

```
$double = createMultiplier(2);
$triple = createMultiplier(3);
```

```
echo $double(5); // Outputs 10
echo $triple(5); // Outputs 15
?>
```

Here, the arrow function captures `$factor` from the parent scope automatically, and the `createMultiplier` function returns the arrow function itself.

Using Arrow Functions in Array Manipulation

Arrow functions are particularly powerful when working with array functions like `array_map()`, `array_filter()`, and `array_reduce()`.

a. Using `array_map()` with Arrow Functions

```
<?php
$numbers = [1, 2, 3, 4, 5];

// Apply multiple operations to each element
$result = array_map(fn($n) => $n * 2 + 1, $numbers);

print_r($result); // Outputs [3, 5, 7, 9, 11]
?>
```

In this example, the arrow function multiplies each element by 2 and then adds 1. It's concise, yet performs multiple operations on the array.

b. Using `array_filter()` with Arrow Functions

```
<?php
$numbers = [10, 15, 20, 25, 30];

// Filter out numbers greater than 20
$filtered = array_filter($numbers, fn($n) => $n > 20);

print_r($filtered); // Outputs [25, 30]
?>
```

The arrow function acts as a predicate to filter only those numbers greater than 20.

c. Using `array_reduce()` with Arrow Functions

```
<?php
$numbers = [1, 2, 3, 4, 5];
```

```
// Calculate the sum of the array
```

```
$sum = array_reduce($numbers, fn($carry, $n) => $carry + $n, 0);
```

```
echo $sum; // Outputs 15
```

```
?>
```

In this example, the arrow function accumulates the sum of the array using `array_reduce()`.

Arrow Functions with Objects

Arrow functions can be used inside object methods to perform concise operations, and they can capture `this` (current object context) from the parent scope.

```
<?php
```

```
class Calculator {
```

```
    private $factor;
```

```
    public function __construct($factor) {
```

```
        $this->factor = $factor;
```

```
    }
```

```
    public function calculate($numbers) {
```

```
        return array_map(fn($n) => $n * $this->factor, $numbers);
```

```
    }
```

```
}
```

```
$calculator = new Calculator(3);
```

```
$numbers = [1, 2, 3, 4, 5];
```

```
print_r($calculator->calculate($numbers)); // Outputs [3, 6, 9, 12, 15]
```

?>

Here, the arrow function captures `$this` (the object itself) and uses `$this->factor` from the parent scope.

Using Arrow Functions in Sorting with `usort()`

Arrow functions can be used for custom sorting logic in `usort()`.

```
<?php
$people = [
    ['name' => 'Alice', 'age' => 25],
    ['name' => 'Bob', 'age' => 30],
    ['name' => 'Charlie', 'age' => 20]
];

// Sort people by age using arrow function
usort($people, fn($a, $b) => $a['age'] <=> $b['age']);

print_r($people);
?>
```

In this case, the arrow function is used as a custom comparison function to sort the array of people by age.

Arrow Functions with Nested Functions

Arrow functions can be nested inside other functions or arrow functions.

```
<?php
$multiplier = fn($factor) => fn($n) => $n * $factor;

$double = $multiplier(2);
$triple = $multiplier(3);

echo $double(5); // Outputs 10
echo $triple(5); // Outputs 15
```


?>

Here, the first arrow function returns another arrow function, demonstrating how you can nest arrow functions to create reusable, concise logic.

Arrow functions are a great way to make your code shorter and more readable for simple expressions. They automatically inherit variables from the parent scope and are perfect for use in callbacks and array manipulations. However, they are limited to single expressions and cannot handle multiple statements.

TOPIC 7 PHP AND MYSQL

PHP PART 2

APPLICATION DEVELOPMENT

Ensure you have installed Xampp

XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]

Service	Module	PID(s)	Port(s)	Actions
☐	Apache	18124 16760	80, 443	Stop Admin Config Logs
☐	MySQL	20836	3306	Stop Admin Config Logs
☐	FileZilla			Start Admin Config Logs
☐	Mercury			Start Admin Config Logs
☐	Tomcat			Start Admin Config Logs

Config Netstat Shell Explorer Services Help Quit

11:52:09 AM [mysql] This may be due to a blocked port, missing dependencies,
11:52:09 AM [mysql] improper privileges, a crash, or a shutdown by another method.
11:52:09 AM [mysql] Press the Logs button to view error logs and check
11:52:09 AM [mysql] the Windows Event Viewer for more clues
11:52:09 AM [mysql] If you need more help, copy and post this
11:52:09 AM [mysql] entire log window on the forums
11:53:45 AM [mysql] Attempting to start MySQL app...
11:53:45 AM [mysql] Status change detected: running

Database: seku Table: students

Server: 127.0.0.1 » Database: seku » Table: students

Browse

Structure

SQL

Search

Insert

Export

Import

Privileges

Table structure

Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 regno 	varchar(20)	utf8mb4_general_ci		No	None			 Change  Drop  More
☐	2 firstname	varchar(50)	utf8mb4_general_ci		No	None			 Change  Drop  More
☐	3 lastname	varchar(50)	utf8mb4_general_ci		No	None			 Change  Drop  More
☐	4 gender	varchar(12)	utf8mb4_general_ci		No	None			 Change  Drop  More

SQL Code

- CREATE DATABASE SEKU;
- CREATE TABLE `students` (
 `regno` varchar(20) NOT NULL,
 `firstname` varchar(50) NOT NULL,
 `lastname` varchar(50) NOT NULL,
 `gender` varchar(12) NOT NULL
);

Connecting to Database Example 1

```
<?php
// Database connection settings
$host = 'localhost';
$user = 'root';
$password = '';
$dbname = 'seku';
// Connect to the database
$conn = new mysqli($host, $user, $password, $dbname);
// Check the connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

Connecting to Database Example 2

```
<?php
```

```
define('DB_SERVER', 'localhost');
```

```
define('DB_USERNAME', 'root');
```

```
define('DB_PASSWORD', '');
```

```
define('DB_NAME', 'seku');
```

```
/* Attempt to connect to MySQL database */
```

```
$conn = mysqli_connect(DB_SERVER, DB_USERNAME, DB_PASSWORD, DB_NAME);
```

```
// Check connection
```

```
if($conn === false){
```

```
    die("ERROR: Could not connect. " . mysqli_connect_error());
```

```
}
```

```
?>
```

Add Students

[View Students](#) [About](#) [Contact](#)

Add Student

Reg No:

First Name:

Last Name:

Reg No:

Select ▼

Save Student

HTML CODE FOR FORM

1. `<h2>Add Student</h2>`
2. `<form method="POST" action="">`
3. `<label>Reg No:</label>`
4. `<input type="text" name="regno" required>`
5. `<label>First Name:</label>`
6. `<input type="text" name="firstname" required>`
7. `<label>Last Name:</label>`
8. `<input type="text" name="lastname" required>`
9. `<label>Reg No:</label>`
10. `<select name="gender" id="gender" required>`
11. `<option value="" selected>Select</option>`
12. `<option value="Male">Male</option>`
13. `<option value="Female">Female</option>`
14. `</select><br><br>`
15. `<button type="submit" name="save">Save Student</button>`
16. `</form>`

Display all Student

[Home](#) [About](#) [Contact](#)

All Students

Reg No	First Name	Last Name		Action
G124/2020/2023	Thoma	Mbingu	Male	Edit Delete
G126/5541/2031	Eric	Thomas	Male	Edit Delete
G127/3210/2030	Emilia	Jones	Female	Edit Delete

Part of Display Code

```
1.  <?php include "myfunctions.php";?>
2.  <html lang="en">
3.  <head>
4.  <title>Student Management</title>
5.      <link rel="stylesheet" type="text/css" href="main.css">
6.      <link rel="stylesheet" type="text/css" href="menu.css">
7.  </head>
8.  <body>
9.  <div class="menu-bar">
10.      <a href="index.php">Home</a>
11.      <a href="about.php">About</a>
12.      <a href="contact.php">Contact</a>
13. </div>
14. <h2>All Students</h2>
15. <?php getStudents(); ?>
16. </body>
17. </html>
18. <?php
19. // Close the database connection
20. $conn->close();
21. ?>
```

Edit Student Records

[Home](#)[About](#)[Contact](#)

Reg No: G124/2020/2023

First Name:

Thoma

Last Name:

Mbingu

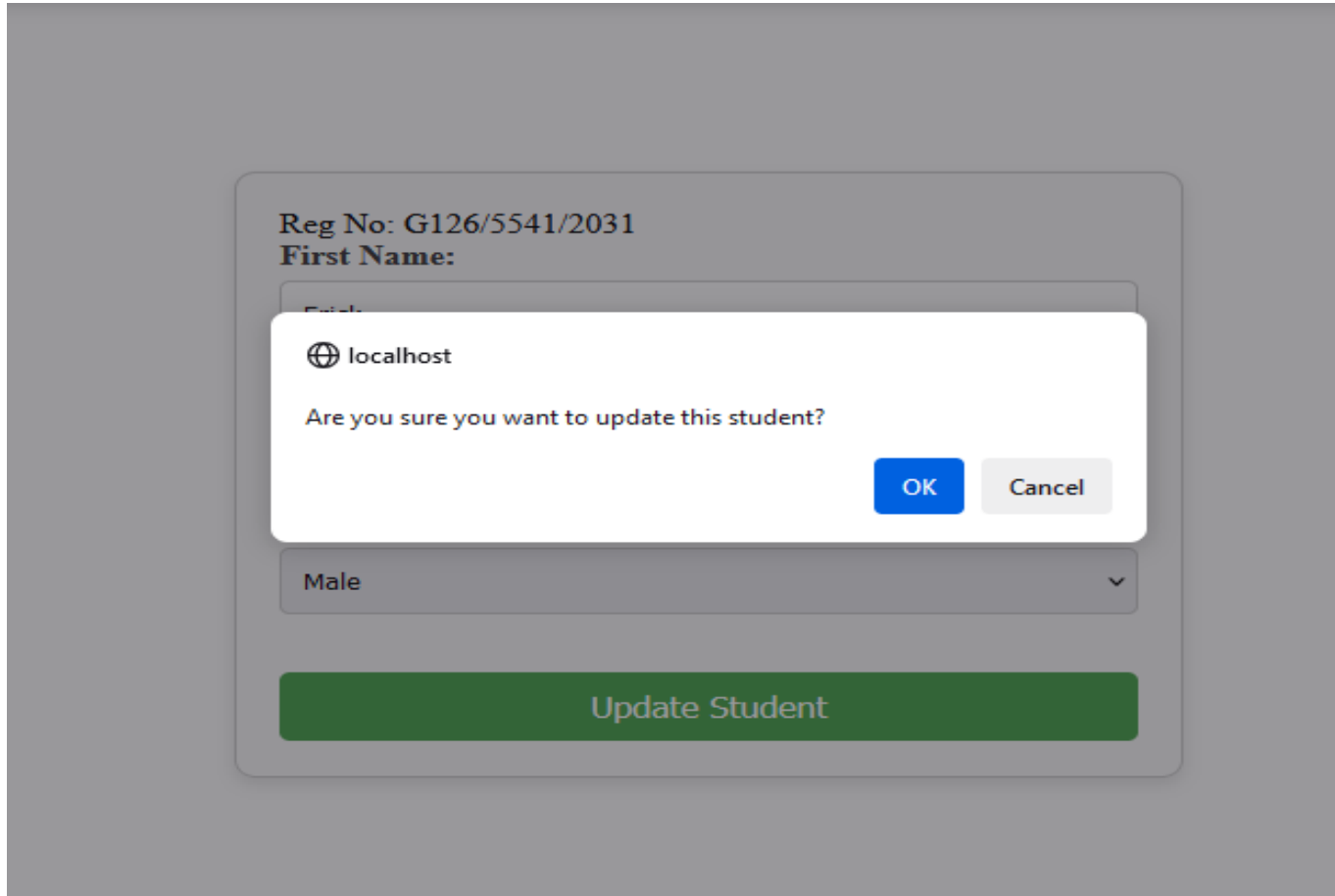
Gender:

Male



Update Student

Edit Student Records



The screenshot displays a web interface for editing student records. At the top, the registration number "Reg No: G126/5541/2031" is shown. Below it, the "First Name:" field is partially visible. A confirmation dialog box is centered on the screen, asking "Are you sure you want to update this student?". The dialog includes a globe icon and the text "localhost". At the bottom of the dialog are "OK" and "Cancel" buttons. Below the dialog, a dropdown menu shows "Male" as the selected gender. At the very bottom is a large green button labeled "Update Student".

Reg No: G126/5541/2031

First Name:

localhost

Are you sure you want to update this student?

OK Cancel

Male

Update Student

Edit code

```
1. <html><head>
    <title>Edit Student</title>
    <link rel="stylesheet" type="text/css" href="edit.css">
    <link rel="stylesheet" type="text/css" href="menu.css">
    </head>

2. <body>

3. <div class="menu-bar">

4.     <a href="index.php">Home</a> <a href="about.php">About</a> <a href="contact.php">Contact</a>

5. </div>

6. <h2>Edit Student</h2>

7. <div class="student-form">

8.     Reg No: <?php echo $student['regno']; ?> <br>

9.     <label>First Name:</label>

10.    <input type="text" id="firstname" value=" <?php echo $student['firstname']; ?>"><br>

11.    <label>Last Name:</label>

12.    <input type="text" id="lastname" value="<?php echo $student['lastname']; ?>"><br>

13.    <label>Gender:</label>

14.    <select id="gender">

15.        <option value="Male" <?php if ($student['gender'] == 'Male') echo 'selected'; ?>>Male</option>

16.        <option value="Female" <?php if ($student['gender'] == 'Female') echo 'selected'; ?>>Female</option>

17.    </select><br><br>

18.    <button onclick="updateStudent('<?php echo $student['regno']; ?>')">Update Student</button>

19. </div>
```

```
22. <script>
23. function updateStudent(regno) {
24.     const firstname = document.getElementById('firstname').value;
25.     const lastname = document.getElementById('lastname').value;
26.     const gender = document.getElementById('gender').value;
27.     if (confirm('Are you sure you want to update this student?')) {
28.         const form = document.createElement('form');
29.         form.method = 'POST';
30.         form.action = "";
31.         form.innerHTML = `
32.             <input type='hidden' name='regno' value='${regno}'>
33.             <input type='hidden' name='firstname' value='${firstname}'>
34.             <input type='hidden' name='lastname' value='${lastname}'>
35.             <input type='hidden' name='gender' value='${gender}'>
36.             <input type='hidden' name='update' value='1'>    `;
37.         document.body.appendChild(form);
38.         form.submit();
39.     }
40. }
41. </script>
42. </body></html>
```

Delete

[Home](#) [About](#) [Contact](#)

All Students

	First Name	Last Name		Action
	Eric	Thomas	Male	Edit Delete
	Emilia		Female	Edit Delete

localhost

Are you sure you want to delete this student?

OK

Cancel

// Handling form submissions and delete actions

1. `if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['save'])) {`
2. `saveStudent($_POST['regno'], $_POST['firstname'],
$_POST['lastname'], $_POST['gender']);`
3. `} elseif (isset($_GET['delete'])) {`
4. `deleteStudent($_GET['delete']);`
5. `}`

// Function to retrieve all students with Delete links

```
1. function getStudents() {
2.     global $conn; // Ensure the connection object is accessible here
3.     $result = $conn->query("SELECT * FROM students");
4.     if ($result->num_rows > 0) {
5.         echo "<table border='1'><tr><th>Reg No</th><th>First Name</th><th>Last
Name</th><th></th><th>Action</th></tr>";
6.         while ($row = $result->fetch_assoc()) {
7.             echo "<tr> <td>{$row['regno']}</td> <td>{$row['firstname']}</td> <td>{$row['lastname']}</td>
8.                 <td>{$row['gender']}</td>
<td><a href='edit.php?regno={$row['regno']}'>Edit</a> | <a href='?delete={$row['regno']}'
onclick='return confirm(\"Are you sure you want to delete this student?\");'>Delete</a></td> </tr>";
9.         }
10.        echo "</table>";
11.    } else {
12.        echo "No students found.<br>";
13.    }
14. }
```

// Function to save student data

```
1. function saveStudent($regno, $firstname, $lastname, $gender) {
2.     global $conn; // Ensure the connection object is accessible here
3.     $stmt = $conn->prepare("INSERT INTO students (regno, firstname, lastname, gender) VALUES (?, ?, ?,
    ?)");
4.     $stmt->bind_param("ssss", $regno, $firstname, $lastname, $gender);

5.     if ($stmt->execute()) {
6.         echo "Student saved successfully!<br>";
7.         header("Location: studentRecords.php"); // Redirect to main page
8.         exit; // Ensure no further code is executed
9.     } else {
10.        echo "Error: " . $stmt->error . "<br>";
11.    }

12.    $stmt->close();
13. }
```

// Function to save student data Example 2

```
function saveStudent($regno, $firstname, $lastname, $gender) {  
    global $conn;  
    $regno = mysqli_real_escape_string($conn, $regno);  
    $firstname = mysqli_real_escape_string($conn, $firstname);  
    $lastname = mysqli_real_escape_string($conn, $lastname);  
    $gender = mysqli_real_escape_string($conn, $gender);  
    $sql = "INSERT INTO students (regno, firstname, lastname, gender)  
        VALUES ('$regno', '$firstname', '$lastname', '$gender')";  
    if (mysqli_query($conn, $sql)) {  
        echo "Student saved successfully!<br>";  
        header("Location: studentRecords.php");  
        exit;  
    } else {  
        echo "Error: " . mysqli_error($conn) . "<br>";  
    }  
}
```

/ Update student details without a form

```
1.  if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['update'])) {
2.      $regno = $_POST['regno'];
3.      $firstname = $_POST['firstname'];
4.      $lastname = $_POST['lastname'];
5.      $gender = $_POST['gender'];
6.      $stmt = $conn->prepare("UPDATE students SET firstname = ?, lastname = ?, gender = ? WHERE regno =
    ?");
7.      $stmt->bind_param("ssss", $firstname, $lastname, $gender, $regno);
8.      if ($stmt->execute()) {
9.          echo "Student updated successfully!<br>";
10.         header("Location: studentRecords.php"); // Redirect to main page
11.         exit; // Ensure no further code is executed
12.     } else {
13.         echo "Error: " . $stmt->error . "<br>";
14.     }
15.     $stmt->close();
16. }
```

- // Function to delete a student by regno

1. function deleteStudent(\$regno) {
2. global \$conn; // Ensure the connection object is accessible here
3. \$stmt = \$conn->prepare("DELETE FROM students WHERE regno = ?");
4. \$stmt->bind_param("s", \$regno);
5. if (\$stmt->execute()) {
6. echo "Student deleted successfully!
";
7. } else {
8. echo "Error: " . \$stmt->error . "
";
9. }
10. \$stmt->close();
11. }

menu.css

```
/* Styles for the menu bar */

.menu-bar {

    background-color: #4CAF50; /* Green background */

    overflow: hidden; /* Clear floats */

    display: flex; /* Use flexbox for alignment */

    justify-content: center; /* Center the menu items */

    padding: 10px 0; /* Top and bottom padding */

    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.2); /* Subtle shadow */

}

.menu-bar a {

    color: white; /* Text color */

    text-align: center; /* Center text */

    padding: 14px 20px; /* Padding for the links */

    text-decoration: none; /* Remove underline */

    transition: background-color 0.3s; /* Smooth transition */

}

.menu-bar a:hover {

    background-color: #45a049; /* Darker green on hover */

}

/* Responsive behavior */

@media (max-width: 600px) {

    .menu-bar {

        flex-direction: column; /* Stack items on small screens */

    }

    .menu-bar a {

        padding: 10px; /* Adjust padding */

    }

}
```

main.css

```
/* Reset some basic elements */
```

```
body {  
  
    margin: 0;  
  
    padding: 0;  
  
    font-family: Arial, sans-serif;  
  
    background-color: #f4f4f4;  
  
    color: #333;  
  
}
```

```
h2 {  
  
    text-align: center;  
  
    color: #4CAF50;  
  
}
```

```
/* Form styling */
```

```
form {  
  
    max-width: 600px;  
  
    margin: 20px auto;  
  
    padding: 20px;  
  
    background: #fff;  
  
    border-radius: 8px;  
  
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);  
  
}
```

```
form label {  
  
    display: block;  
  
    margin: 10px 0 5px;  
  
}
```


main.css

```
form label {  
    display: block;  
    margin: 10px 0 5px;  
}  
  
form input[type="text"],  
  
form select {  
    width: 100%;  
    padding: 10px;  
    margin-bottom: 20px;  
    border: 1px solid #ccc;  
    border-radius: 4px;  
    box-sizing: border-box;  
}  
  
form button {  
    background-color: #4CAF50;  
    color: white;  
    padding: 10px;  
    border: none;  
    border-radius: 4px;  
    cursor: pointer;  
    font-size: 16px;  
}  
  
form button:hover {  
    background-color: #45a049;  
}
```

main.css

```
/* Table styling */
```

```
table {  
  
    width: 100%;  
  
    border-collapse: collapse;  
  
    margin: 20px auto;  
  
    background: #fff;  
  
}  
  
table th,  
table td {  
  
    padding: 12px;  
  
    text-align: left;  
  
    border-bottom: 1px solid #ddd;  
  
}  
  
table th {  
  
    background-color: #4CAF50;  
  
    color: white;  
  
}  
  
table tr:hover {  
  
    background-color: #f1f1f1;  
  
}  
  
table a {  
  
    color: #007BFF;  
  
    text-decoration: none;  
  
}  
  
table a:hover {  
  
    text-decoration: underline;  
  
}
```

edit.css

```
/* Styling the container */
.student-form {
  max-width: 400px;
  margin: 50px auto;
  padding: 20px;
  border: 1px solid #ccc;
  border-radius: 10px;
  background-color: #f9f9f9;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
}

/* Labels and Inputs alignment */
.student-form label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
  color: #333;
}

.student-form input[type="text"],
select {
  width: 100%;
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 5px;
  box-sizing: border-box;
}
```

edit.css

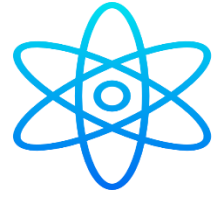
```
/* Styling the button */
```

```
.student-form button {  
  width: 100%;  
  padding: 10px;  
  background-color: #4CAF50;  
  color: white;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
  font-size: 16px;  
  transition: background-color 0.3s ease;  
}  
  
.student-form button:hover {  
  background-color: #45a049;  
}
```

```
/* Responsive Design */
```

```
@media (max-width: 500px) {  
  .student-form div {  
    width: 90%;  
    margin: 20px auto;  
  }  
}
```

TOPIC 8 INTRODUCTION TO REACT.JS



REACT TUTORIAL

- React is a **JavaScript library** for building user interfaces, particularly for single-page applications where updates to the UI need to be fast and dynamic.
- It was developed by Facebook and released as open source in 2013.
- React allows developers to build web applications that can update and render efficiently based on data changes.

Key Features of React:

1. **Component-Based Architecture:** React applications are built using components, which are reusable and independent pieces of the user interface.
2. **Declarative Syntax:** React uses a declarative style of programming, making code easier to read and debug. Developers describe **what** the UI should look like, and React takes care of the **how**.
3. **Virtual DOM:** React uses a virtual DOM (Document Object Model) to optimize updates to the real DOM.

This improves performance by minimizing direct DOM manipulations.

4. **JSX:** React introduces JSX (JavaScript XML), a syntax extension that allows HTML-like code to be written within JavaScript files, making the code more intuitive.
5. **Unidirectional Data Flow:** Data flows in one direction (from parent to child components), which simplifies state management and debugging.
6. **Rich Ecosystem:** React has a strong ecosystem with tools like React Router for navigation, Redux for state management, and Next.js for server-side rendering.

Why Use React?

- **Speed and Efficiency:** React's virtual DOM and component system make it fast and scalable.
- **Flexibility:** React can be used for web apps, mobile apps (via React Native), and even desktop apps.
- **Community Support:** It has a massive community and an extensive library of resources and tools.

JSX CODE:

- JSX (JavaScript XML) is a syntax extension for JavaScript that allows you to write HTML-like code directly within JavaScript.
- It makes it easier to create and understand the structure of a React component.
- JSX combines JavaScript logic and markup in the same file, enabling a more intuitive approach to building user interfaces.

Key Features of JSX:

1. **HTML-like Syntax:** It looks like HTML but is written within JavaScript files.
2. **JavaScript Inside JSX:** You can embed JavaScript expressions within {} braces.
3. **Compiles to JavaScript:** JSX is transpiled into plain JavaScript using tools like Babel.

```
import React from 'react';

function Greeting() {
  const name = "Alice";

  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>Welcome to learning React with JSX.</p>
    </div>
  );
}

export default Greeting;
```

OUTPUT:

Hello, Alice!

Welcome to learning React with JSX.

- **Import React:** React must be imported to use JSX.

- **Component:** The Greeting function is a React component that returns JSX.
- **Embedding JavaScript:** {name} inside the <h1> tag dynamically inserts the value of the JavaScript variable name.
- **Rendering HTML Elements:** JSX allows you to define the UI structure (<h1> and <p> tags).

Why Use JSX?

- **Readability:** JSX makes the code easier to read by closely resembling HTML.
- **Logic + UI Together:** You can mix JavaScript logic with UI markup seamlessly.
- **Reusable Components:** JSX works perfectly for defining and reusing UI components in React.

STEPS TO WORKING WITH REACT JS

1 Set Up the Environment: Use a tool like Vite or Create React App to scaffold a React project quickly. These tools handle the boilerplate setup for you, including a pre-configured development server and build tools.

2. Understand the Basics of JSX: JSX is a syntax extension for JavaScript that allows you to write HTML-like code within JavaScript. For example:

```
1 function App() {  
2   return <h1>Hello, World!</h1>;  
3 }
```

3.Create Components: React revolves around reusable components. A component is a JavaScript function that returns JSX. For example:

```
1 function Greeting() {  
2   return <p>Welcome to React!</p>;  
3 }
```


4 Use State and Props: Components in React can have state (data managed within the component) and props (data passed between components). Learn how to manage dynamic data with `useState`:

```
1 import { useState } from 'react';
2
3 function Counter() {
4   const [count, setCount] = useState(0);
5   return <button onClick={() => setCount(count + 1)}>Count: {count}</button>;
6 }
```

5. Learn Event Handling: Handle user interactions like clicks, input changes, and form submissions by defining event handlers directly in JSX:

```
1 function App() {
2   function handleClick() {
3     alert("Button clicked!");
4   }
5   return <button onClick={handleClick}>Click Me</button>;
6 }
7
```

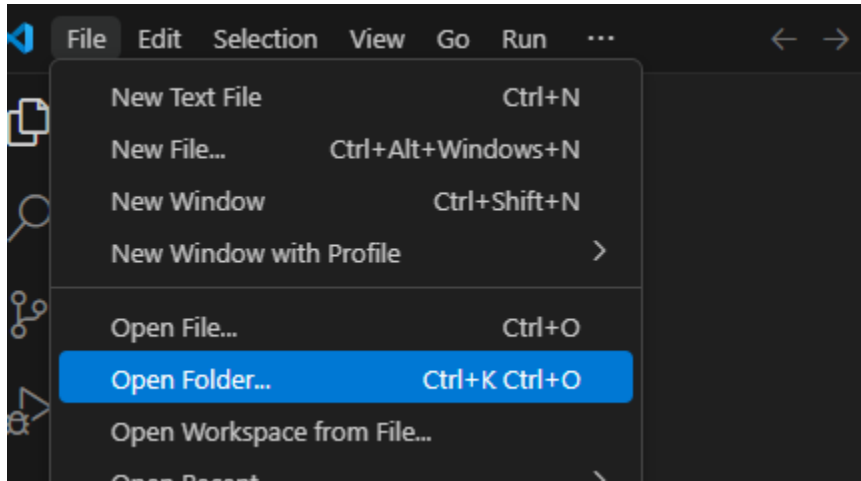
6. Rendering Lists and Conditional Logic: Use JavaScript features like `.map()` for lists and `if` or the conditional (`?`) operator for dynamic rendering:

```
1 function ItemList({ items }) {
2   return (
3     <ul>
4       {items.map((item) => (
5         <li key={item.id}>{item.name}</li>
6       ))}
7     </ul>
8   );
9 }
```

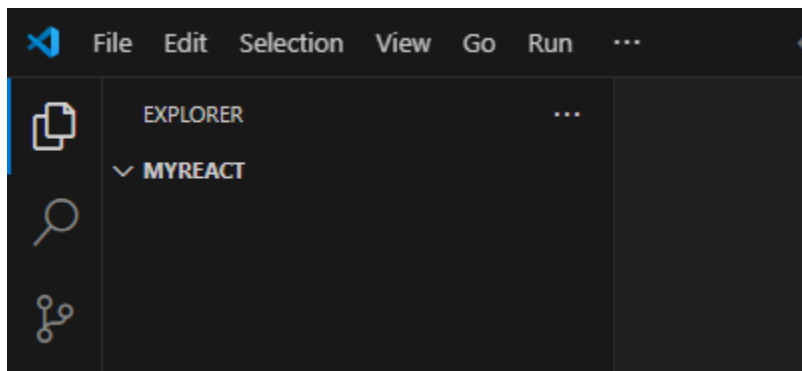
CREATING A SIMPLE APPLICATION TO OUTPUT SEKU ON A WEB PAGE

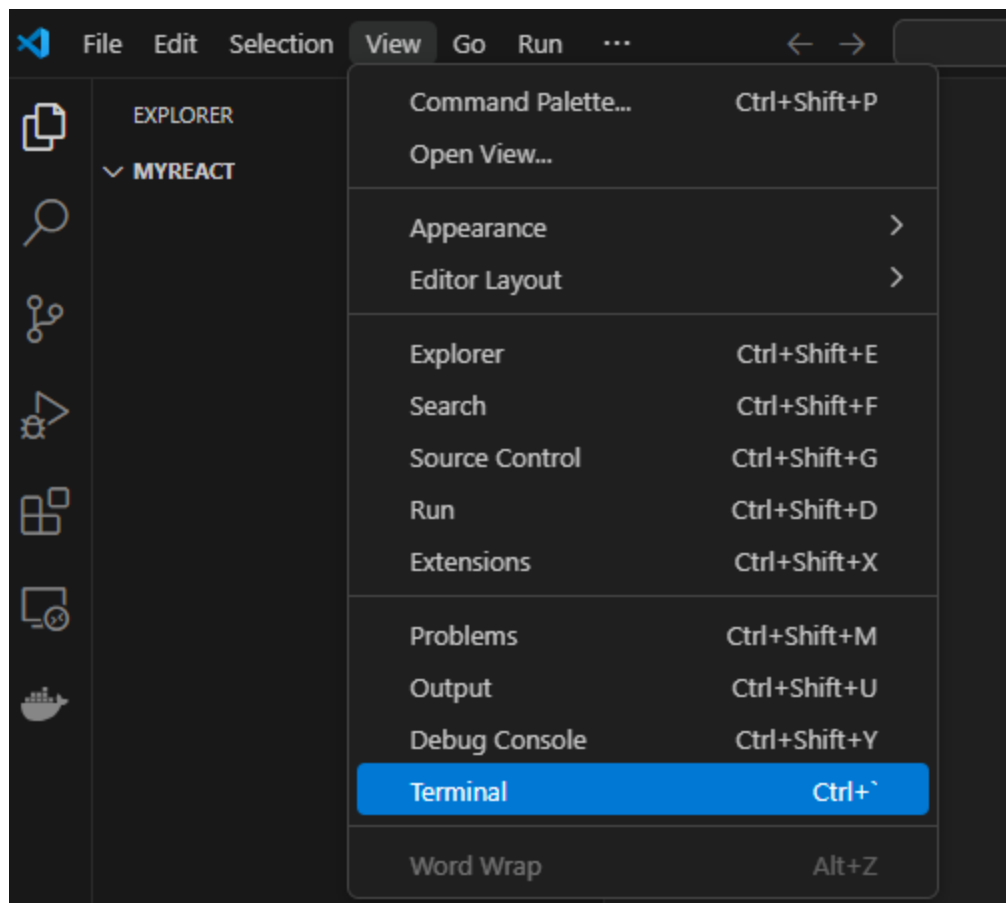
INSTALL VSCode

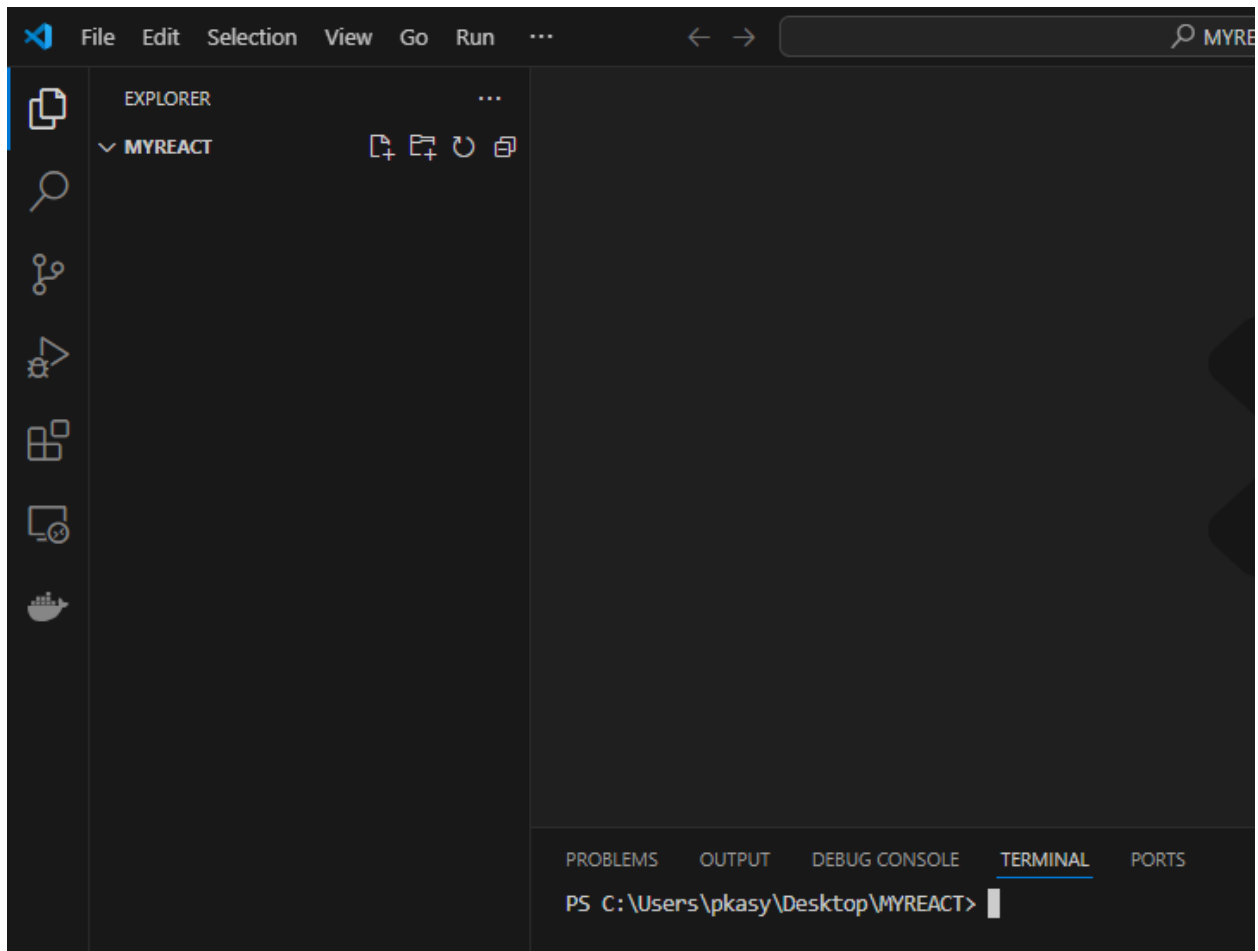
After installing VSCode goto File select Open Folder {You can create a new folder or open an existing folder:



We have created folder MYREACT







To create our application we will use Vite

Vite is a modern build tool designed for front-end web development, offering a faster and more efficient development experience compared to traditional tools like Webpack or Create React App. It is particularly useful in the following scenarios:

1. Development Speed:

2. Lightweight Projects:

3. React, Vue, Svelte, or Vanilla JavaScript:

- Vite has excellent support for modern frameworks like React, Vue, and Svelte. It also works seamlessly with plain JavaScript projects.

4. Optimal Production Builds:

- For production, Vite uses **Rollup**, a highly optimized bundler, to create efficient and optimized builds.
- **When to use:** You need fast and optimized builds for deployment.

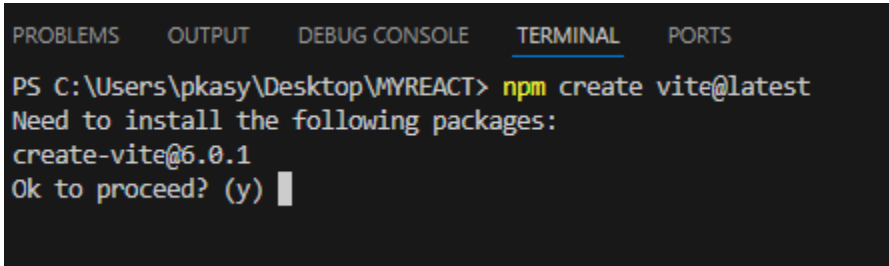
5. Modern Features:

- Vite supports modern JavaScript features like ES6 modules, dynamic imports, and TypeScript without additional plugins.

6. Reducing Tooling Overhead:

- Vite has minimal configuration requirements and offers built-in features like a development server, file-based routing, and easy integration with plugins.

To Install Vite use command : `npm create vite@latest`

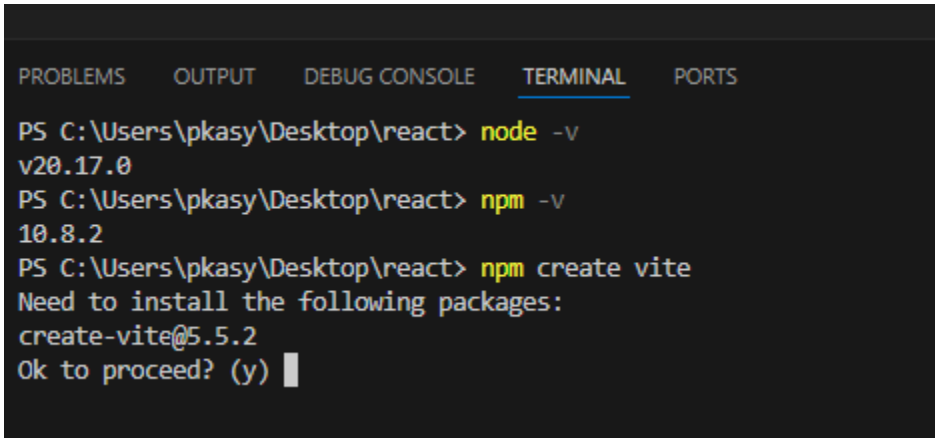
A screenshot of a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The prompt is 'PS C:\Users\pkasy\Desktop\MYREACT>'. The command entered is 'npm create vite@latest'. The output shows 'Need to install the following packages:', 'create-vite@6.0.1', and 'Ok to proceed? (y)' with a cursor.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\pkasy\Desktop\MYREACT> npm create vite@latest
Need to install the following packages:
create-vite@6.0.1
Ok to proceed? (y)

```

Install VSCode, node.js

A screenshot of a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The prompt is 'PS C:\Users\pkasy\Desktop\react>'. The commands entered are 'node -v', 'npm -v', and 'npm create vite'. The output shows 'v20.17.0', '10.8.2', 'Need to install the following packages:', 'create-vite@5.5.2', and 'Ok to proceed? (y)' with a cursor.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\pkasy\Desktop\react> node -v
v20.17.0
PS C:\Users\pkasy\Desktop\react> npm -v
10.8.2
PS C:\Users\pkasy\Desktop\react> npm create vite
Need to install the following packages:
create-vite@5.5.2
Ok to proceed? (y)

```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\pkasy\Desktop\MYREACT> npm create vite@latest
Need to install the following packages:
create-vite@6.0.1
Ok to proceed? (y) y

> npx
> create-vite

? Project name: » vite-project
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\pkasy\Desktop\MYREACT> npm create vite@latest
Need to install the following packages:
create-vite@6.0.1
Ok to proceed? (y) y

> npx
> create-vite

? Project name: » studentsApp
```

Select React

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

✓ Project name: ... studentsApp
✓ Package name: ... studentsapp
? Select a framework: » - Use arrow-keys. Return to submit.
  Vanilla
  Vue
>  React
  Preact
  Lit
  Svelte
  Solid
  Qwik
  Angular
  Others
```

Select y for yes then assign a project name and select framework

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

✓ Project name: ... reacttutorial
? Select a framework: » - Use arrow-keys. Return to submit.
> Vanilla
  Vue
  React
  Preact
  Lit
  Svelte
  Solid
  Qwik
  Others
```

Select JavaScript

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

✓ Project name: ... studentsApp
✓ Package name: ... studentsapp
✓ Select a framework: » React
? Select a variant: » - Use arrow-keys. Return to submit.
  TypeScript
  TypeScript + SWC
> JavaScript
  JavaScript + SWC
  React Router v7 ↗
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

✓ Project name: ... studentsApp
✓ Package name: ... studentsapp
✓ Select a framework: » React
✓ Select a variant: » JavaScript

Scaffolding project in C:\Users\pkasy\Desktop\MYREACT\studentsApp...

Done. Now run:

  cd studentsApp
  npm install
  npm run dev

PS C:\Users\pkasy\Desktop\MYREACT>
```

Follow the above instructions starting with `cd studentsApp` then `npm install`

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

✓ Project name: ... studentsApp
✓ Package name: ... studentsapp
✓ Select a framework: » React
✓ Select a variant: » JavaScript

Scaffolding project in C:\Users\pkasy\Desktop\MYREACT\studentsApp...

Done. Now run:

  cd studentsApp
  npm install
  npm run dev

PS C:\Users\pkasy\Desktop\MYREACT> cd studentsApp
PS C:\Users\pkasy\Desktop\MYREACT\studentsApp> npm install
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

cd studentsApp
npm install
npm run dev

PS C:\Users\pkasy\Desktop\MYREACT> cd studentsApp
PS C:\Users\pkasy\Desktop\MYREACT\studentsApp> npm install

added 250 packages, and audited 251 packages in 4m

102 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\pkasy\Desktop\MYREACT\studentsApp> npm run dev
```

Use npm run dev to run the application

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

VITE v6.0.3  ready in 3687 ms

→ Local:  http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Go to browser and run <http://localhost:5173>



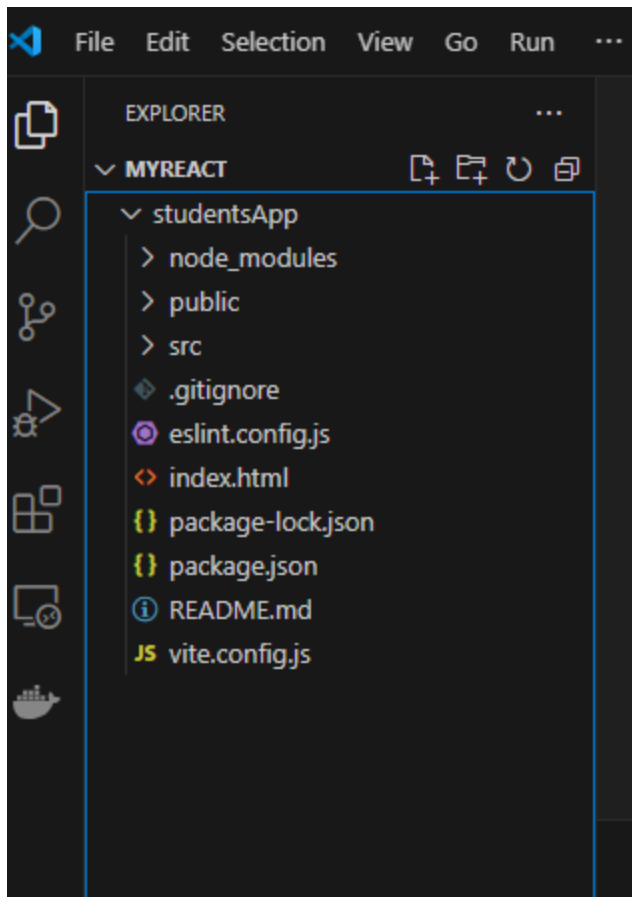
localhost:5173



Vite + React

count is 0

Edit `src/App.jsx` and save to test HMR



1. node_modules/

- **Purpose:** This folder contains all the dependencies (libraries and packages) required by the project.
- **Details:**
 - It is automatically created when you run npm install
 - Contains both the dependencies listed in package.json and their transitive dependencies.
 - **Important:** You generally don't edit anything inside node_modules directly. It's a generated folder and can be recreated from package.json.

2. public/

- **Purpose:** Holds static assets and the main HTML file for the application.

- **Details:**
 - The index.html file inside this folder serves as the entry point for the React app.
 - Any files in public are served as-is without processing by the bundler.
 - **Examples:**
 - Favicon icons (favicon.ico).
 - Static assets like images or robots.txt.
 - To reference files here, you typically use / or %PUBLIC_URL% in paths.

3. src/

- **Purpose:** Contains the application code written in React.
- **Details:**
 - This is where you write all your components, styles, hooks, and logic.
 - **Key file:** src/index.js or src/index.tsx is the starting point where the React application is initialized and rendered into the DOM.
 - The bundler processes everything in src/ (e.g., transpiling JavaScript/TypeScript, bundling assets).

4. package.json

- **Purpose:** Manages the metadata, dependencies, and scripts for the project.
- **Details:**
 - Lists the project dependencies (e.g., react, react-dom) and their required versions.
 - Contains **scripts** to run the project, such as npm start (starts a development server) or npm build (builds the project for production).
 - Includes metadata like the project name, version, description, and author.
 - Acts as the central file to configure and track the project.

5. package-lock.json

- **Purpose:** Locks the exact versions of dependencies for consistent installations.
- **Details:**
 - Ensures all developers or deployment environments use the same versions of packages and their sub-dependencies.
 - Automatically updated when you install a new dependency or update one.

- Helps with debugging and maintaining consistency by specifying dependency trees in detail.

COMPONENTS IN REACT

React Components

Components are independent and reusable bits of code. They serve the same purpose as JavaScript functions, but work in isolation and return HTML via a `render()` function.

Components come in two types:

- Class components and
- Function components

1. Class Component

- Functional components are simpler than class components in React. They extend the `React.Component` class.
- Class components in react contain the following essential features:
 1. **State:** It contains information that may vary over time and has an impact on how a component renders. by way of `this.state`.
 2. **Props:** Data transmitted by a parent component. They serve as a read-only data and configuration route for child components. Accessible through `this.props`.
 3. **Lifecycle Methods:** Activities carried out at particular phases of the component's lifecycle. `ComponentDidMount()`, `ComponentDidUpdate()`, and `ComponentWillUnmount()` are a few examples.
 4. **Event Handlers:** Procedures that specify and manage events brought on by user activities. [JSX](#)-based attachments to elements, such as `onClick={this.handleClick}`.
 5. **Class Methods:** Additional, customized methods that are a part of the component's class and carry out the particular logic or tasks that the component requires.
 6. **Render Method:** Provides JSX code describing the structure and content of the component. When the part has to be modified, this method is called and only one root element is returned

```
import React from 'react';

class MyComponent extends React.Component {
  render() {
    return (
      <div>
        <h1>Hello, World!</h1>
      </div>
    );
  }
}
```

Example 2:

```
import React,{Component} from 'react'

class Example extends Component{
  render(){
    return <div>React is Great</div>
  }
}
```

Class component is not used any more in react

2. Functional Components

- The most basic type of React components is functional ones.
- They are JavaScript functions that accept the input [props](#) (short for properties) and return a React element.

- Because they lack an internal state, functional components in react are sometimes known as stateless components.
- They are the only ones in charge of rendering the user interface using the input props.

Here is a simple example of a React functional component:

Example 1 Normal function

```
import React from 'react';

function App(){
  return(
    <div>
      <h1>Hello</h1>
    </div>
  )}

export default App
```

Example 2 Arrow function

```
import React from 'react'

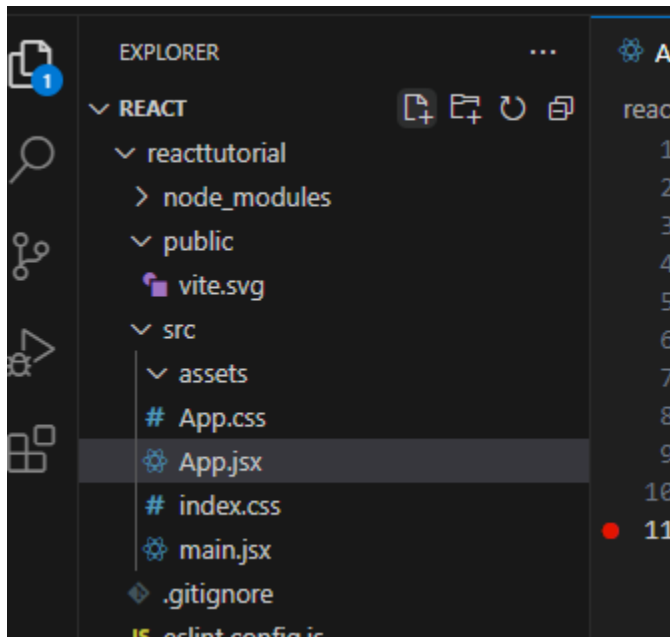
class Example =()=>{
  return(
    <div>React is Great</div>
  )

}

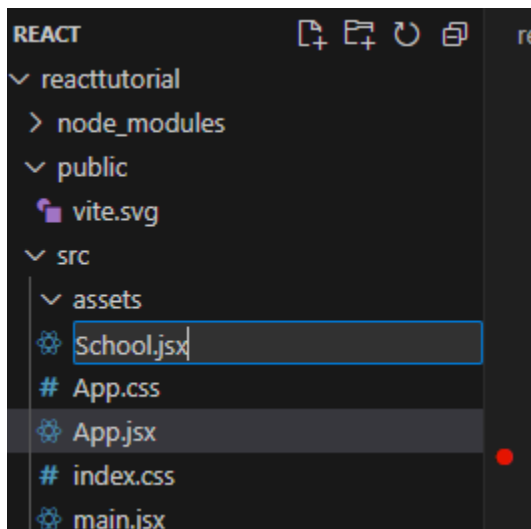
export default Example;
```

Creating a new component:

Name must start in uppercase



We can call it School.jsx



School.jsx

```
import React from 'react'
```

```
const School = () => {
```

```
  return (
```



```

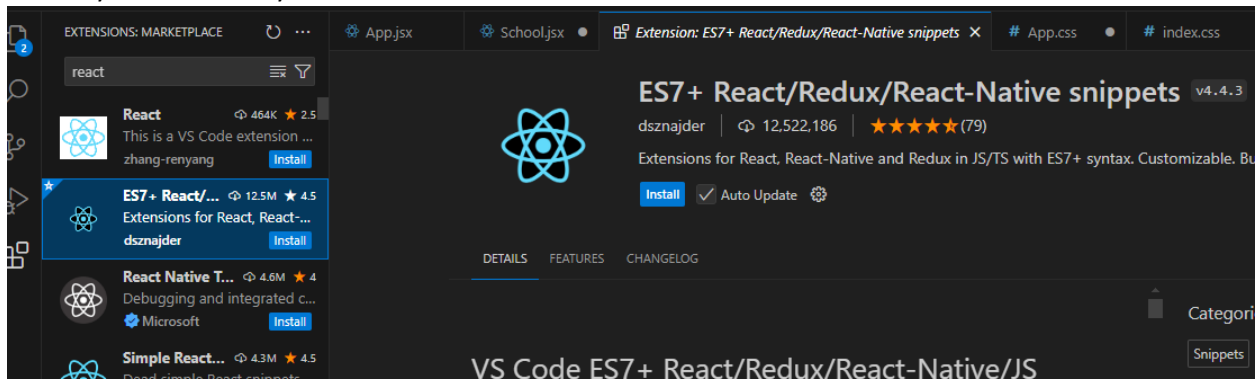
    <div>School</div>
  )
}

export default School

```

NB:

To easily code in react you add extension such as



Instead of typing the whole code above you will only need to type **rafce** which stands for **ReactArrowFunctionExportComponent** and react will autocomplete the code

Exporting:

We have named export and default export

If we want to use the Student.jsx function in App.jsx we can import it

1) **Named export**

Change the Student.jsx component to:

```

export const School = () => {
  return (
    <div>School</div>
  )
}

```

Goto **App.jsx** and import as

```
import {School} from './School' //imported
function App(){
  return(
    <div>
      <h1>Hello</h1>
      <School/> //use it
    </div>
  )
}
export default App
```

2. Default export

School.jsx

```
const School = () => {
  return (
    <div>School</div>
  )
}
export default School
```

App.jsx

```
import School from './School'

function App(){
  return(
    <div>
      <h1>Hello</h1>
      <School/>
    </div>
  )
}
export default App
```

Working with Props

Conceptually, components are like JavaScript functions. They accept arbitrary inputs (called “props”) and return React elements describing what should appear on the screen.

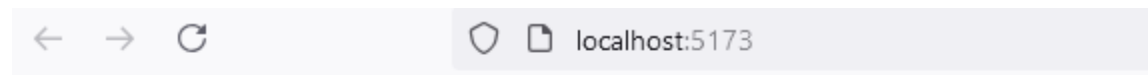
If we wish to pass a message from App.jsx to School.jsx

App.jsx

```
import School from './School'
function App(){
  return(
    <div>
      <h1>Hello</h1>
      <School sname="ICT"/>
    </div>
  )
}
export default App
```

School.jsx

```
const School = (props) => {
  return (
    <div>School {props.sname}</div>
  )
}
export default School
```



Hello

School ICT

Passing variable

App.jsx

```
import School from './School'
function App(){
  const sch="BUSINESS"
  return(
    <div>
      <h1>Hello</h1>
      <School sname={sch}/>
    </div>
  )
}
export default App
```



localhost:5173

Hello

School BUSINESS

Passing an object

App.jsx

```
import School from './School'
```

```
function App(){  
  const schoolInfo={year :2010, name:"ICT"}  
  return(  
    <div>  
      <h1>Hello</h1>  
      <School sch={schoolInfo}/>  
    </div>  
  )  
}  
export default App
```

School.jsx

```
const School = (props) => {  
  return (  
    <div>School {props.sch.year} {props.sch.name}</div>  
  )  
}  
export default School
```



localhost:5173

Hello

School 2010 ICT

Example 2:

App.jsx

This is the parent component where we will define the message "SEKU" and pass it as a prop to the School.jsx component.

```
import React from 'react';
import School from './School';

function App() {
  const message = "SEKU";
  return (
    <div>
      <h1>Welcome to South Eastern Kenya University</h1>
      {/* Passing the message "SEKU" as a prop to School component */}
      <School schoolName={message} />
    </div>
  );
}

export default App;
```

School.jsx

This is the child component that will receive the message (via props) from App.jsx.

```
import React from 'react';

function School({ schoolName }) {
  return (
    <div>
      <h2>School Name:</h2>
      <p>{schoolName}</p>
    </div>
  );
}
```

```

);
}

export default School;

```

Destructuring props

makes your child components' code more readable and makes it easier to see which props are being used at a glance.

App.jsx

```

import School from './School'
function App(){
  const schoolInfo={year :2010, name:"ICT"}
  return(
    <div>
      <h1>Hello</h1>
      <School year={schoolInfo.year} name={schoolInfo.name}/>
    </div>
  )
}
export default App

```

School.jsx

```

const School = ({year, name}) => {
  return (
    <div>School {year} {name}</div>
  )
}
export default School

```

You can get more output by modifying the App.jsx as follows:

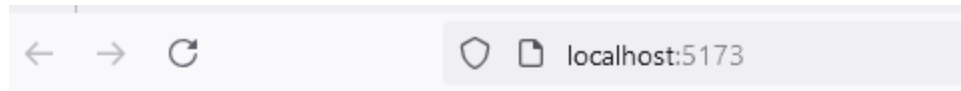
```

import School from './School'

function App(){
  const schoolInfo={year :2010, name:"ICT"}
  const pschoolInfo={year :2014, name:"Pure science"}
  const lschoolInfo={year :2022, name:"Law"}
  return(
    <div>
      <h1>Hello</h1>
      <School year={schoolInfo.year} name={schoolInfo.name}/>
      <School year={pschoolInfo.year} name={pschoolInfo.name}/>
    </div>
  )
}

```

```
<School year={schoolInfo.year} name={lschoolInfo.name}/>
</div>
})
export default App
```



Hello

School 2010 ICT
School 2014 Pure science
School 2010 Law

The best approach would be to use an array of objects but first we need to look at a Simple example of array

```
function App(){
  const array=[10,20,30,40,50,60,70,80,90,100]

  return(
    <div>
    <h1>Rendering array List</h1>
    {
      array.map((number)=>(
        <p>{number}</p>
      ))
    }
    </div>
  )
}
export default App
```



localhost:5173

Rendering array List

10
20
30
40
50
60
70
80
90
100

Now we can use an array of objects to solve the same problem:

App.jsx

```
import School from './School'
```

```
function App(){
```

```
  const schoolInfo=[{year :2010, name:"ICT"},  
                    {year :2014, name:"Pure science"},  
                    {year :2022, name:"Law"}]
```

```
  return(  
    <div>
```

```
    <h1>Rendering the List</h1>
```

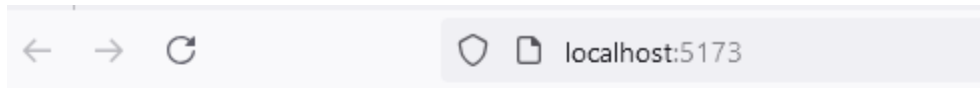
```
    {
```

```
      schoolInfo.map((shule)=>{
```

```
        <School year={shule.year} name={shule.name}/>
```



```
    ))  
  }  
</div>  
})  
export default App
```



Rendering the List

School 2010 ICT
School 2014 Pure science
School 2022 Law

Conditional rendering: Rendering based on a certain occurrence

Conditional rendering in React allows you to render different UI elements based on certain conditions or events

App.jsx

```
function App(){  
  const isLogin=true;  
  if(isLogin){  
    return(  
      <h1>Logged In</h1>  
    )  
  }else{  
    return(  
      <h1>Not Logged In</h1>  
    )  
  }  
}  
export default App
```

using ternary

App.jsx

```
function App(){  
  const isLogin=true;
```

```

if(isLogin){
  return(
    <div>
      {isLogin ? <h1>Logged In</h1> : <h1>Not logged In</h1>}
    </div>
  )
}
}
export default App

```

Working with Hooks

Hooks allow function components to have access to state and other React features. Because of this, class components are generally no longer needed. *Hooks* are a new addition in React 16.8

In React, there are several types of built-in hooks to handle different behavior of the components:

1. State Hooks:

- **‘useState’**: It is the most commonly used React Hook. It allows functional components to have state variables. It takes an initial state value as the argument and returns an array with two elements — the current state value and a function to update that state.

```

import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

```

2. Effect Hooks:

- **‘useEffect’**: It enables performing side effects, such as data fetching, subscriptions, or DOM manipulations after the component has been rendered.

```

import React, { useState, useEffect } from 'react';

function Example() {
  const [data, setData] = useState(null);

```

3. Context Hooks:

- **‘useContext’**: This hook in React is used to consume values from a React context. It allows functional components to access the value provided by a context provider higher up in the component tree without the need for prop drilling.

```

import React from 'react';

// Create a new context
const MyContext = React.createContext();

```

4. Ref Hooks:

- **'useRef'**: Provides a way to create mutable references to values or DOM elements that persist across renders. Often used for accessing or manipulating DOM elements.

```
import React, { useRef } from 'react';
```

5. Callback Hooks:

- **'useCallback'**: Memoizes a callback function, preventing it from being recreated on every render if the dependencies remain unchanged. Useful for optimizing performance by avoiding unnecessary re-renders of child components.

```
'useCallback': Memoizes a callback function, preventing it from being recreated
```

6. Layout Hooks:

- **'useDimensions'**: It measures the dimensions of an element.
- **'useWindowSize'**: It retrieves the current window size.

7. Form Hooks:

- **'useForm'**: It provides form state and validation helpers.

8. Animation Hooks:

- **'useSprings'**: Creates animated values for animation using React Spring.

9. Custom Hooks:

- You can create your own custom hooks to encapsulate reusable logic and stateful behavior. Custom hooks should follow the naming convention of starting with “use” to ensure they can leverage other hooks.

EXAMPLES

1. Using useState Hook

The useState hook lets you add state to functional components.

```
import React, {useState} from 'react';
import School from './School';

function App() {
  const [count, setCount] = useState(0);

  return (
```

```
    <div>

      <p>Count: {count}</p>

      <button onClick={() => setCount(count + 1)}>Increment</button>

      <button onClick={() => setCount(count - 1)}>Decrement</button>

    </div>

  );
}

export default App;
```

2. Using useEffect Hook

The useEffect hook lets you perform side effects, such as fetching data or updating the document title.

Example: Changing Document Title

```
import React, {useState, useEffect} from 'react';
import School from './School';

function App() {
  const [title, setTitle] = useState('React App');
  useEffect(() => {
    document.title = title;
  }, [title]); // Re-run effect only if `title` changes
  return (
    <div>
      <input
        type="text"
        value={title}
```

```
        onChange={(e) => setTitle(e.target.value)}
        placeholder="Change document title"
      />
    </div>
  );
}
export default App;
```

3. Using useReducer Hook

The useReducer hook is useful for managing complex state logic.

```
import React, {useReducer} from 'react';
const initialState = 0;
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return state + 1;
    case 'decrement':
      return state - 1;
    default:
      return state;
  }
}
```

```

function App() {
  const [count, dispatch] = useReducer(reducer, initialState);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>Increment</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>Decrement</button>
    </div>
  );
}

export default App;

```

4. Using useRef Hook

The useRef hook provides a mutable reference that persists across renders

```

import React, { useRef } from 'react';

function App() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus();
  };
}

```

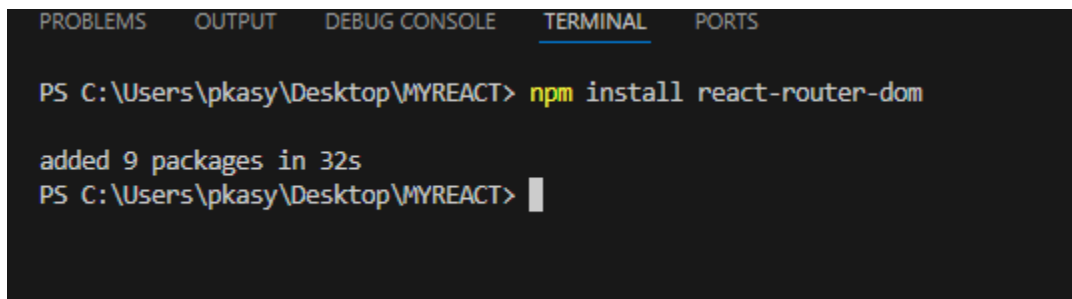
```
return (  
  <div>  
    <input ref={inputRef} type="text" placeholder="Click the button to focus me" />  
    <button onClick={focusInput}>Focus Input</button>  
  </div>  
);  
}  
export default App;
```

Working with react-router-dom

How to Install react-router-dom:

1. Open your terminal and navigate to your React project directory.
2. Run the following command:

npm install react-router-dom



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS C:\Users\pkasy\Desktop\MYREACT> npm install react-router-dom  
  
added 9 packages in 32s  
PS C:\Users\pkasy\Desktop\MYREACT> |
```

After Installation:

- Import the components you need from react-router-dom, such as BrowserRouter, Routes, Route, and Link.

Example

```
import React from 'react';

import { BrowserRouter as Router, Route, Routes, Link } from 'react-router-dom';

function Home() {
  return <h1>Home Page</h1>;
}

function Students() {
  return <h1>Students Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}

function App() {
  return (
    <Router>
      <nav>
        <Link to="/">Home</Link>
        { ' | ' }
        <Link to="/students">Students</Link>
      </nav>
    </Router>
  );
}
```



```
    {' | '}  
    <Link to="/about">About</Link>  
  </nav>  
  <Routes>  
    <Route path="/" element={<Home />} />  
    <Route path="/students" element={<Students />} />  
    <Route path="/about" element={<About />} />  
  </Routes>  
</Router>  
);  
}
```

```
export default App;
```

[Home](#) | [Students](#) | [About](#)

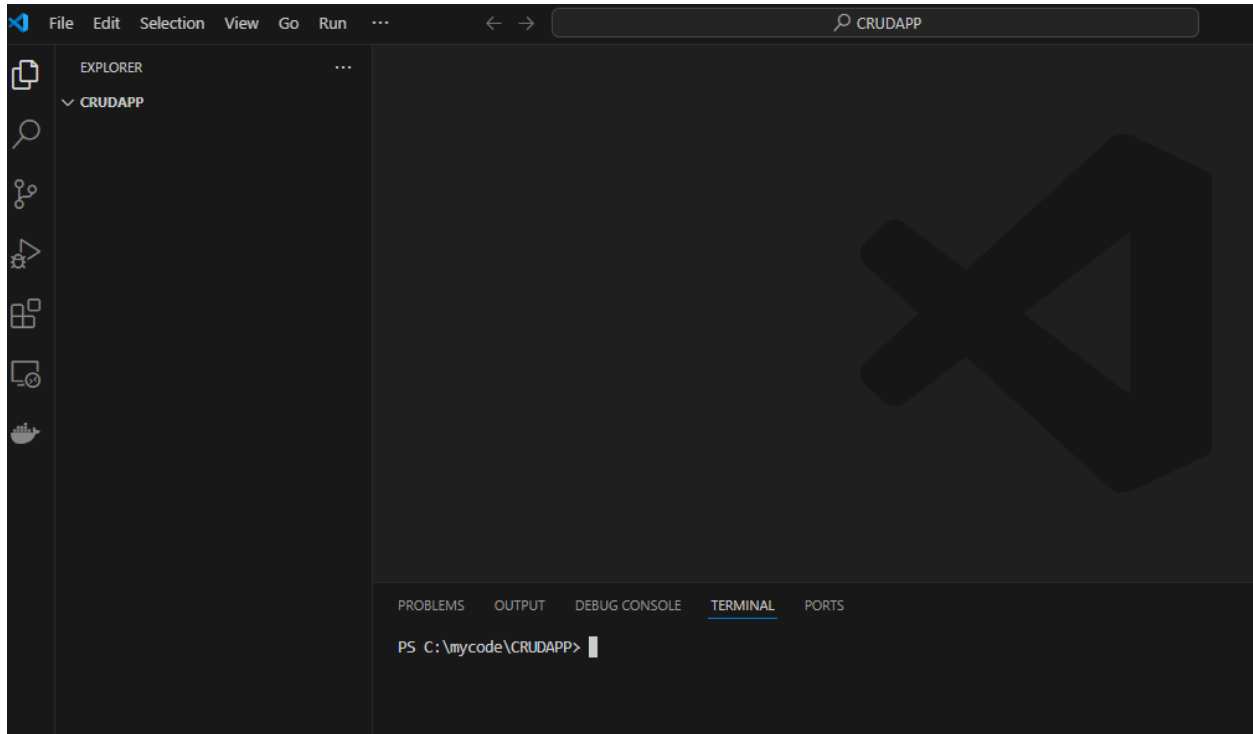
Home Page

TOPIC 9_10 REACT.JS CRUD OPERATIONS

STUDENT REGISTRATION APPLICATION

|

CREATING REACT APP



Create vite

```
PS C:\mycode\CRUDAPP> npm create vite@latest
Need to install the following packages:
create-vite@5.5.5
Ok to proceed? (y) y

> npx
> create-vite

? Project name: » vite-project
```

Type project name: we are going to type client

```
> Vanilla
Vue
React
Preact
Lit
Svelte
Solid
Qwik
Angular
Others
```

Select react and press enter

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

✓ Select a framework: » React
? Select a variant: » - Use arrow-keys. Return to submit.
> TypeScript
  TypeScript + SWC
  JavaScript
  JavaScript + SWC
  Remix ↗
```

Next select Javascript

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

✓ Select a framework: » React
✓ Select a variant: » JavaScript

Scaffolding project in C:\mycode\CRUDAPP\client...

Done. Now run:

  cd client
  npm install
  npm run dev

PS C:\mycode\CRUDAPP> |
```

Select client

Done. Now run:

```
cd client
npm install
npm run dev
```

```
PS C:\mycode\CRUDAPP> cd client
```

Install : dependencies and packages

```
PS C:\mycode\CRUDAPP> cd client
PS C:\mycode\CRUDAPP\client> npm install
```

```
PS C:\mycode\CRUDAPP\client> npm install

added 248 packages, and audited 249 packages in 3m

102 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\mycode\CRUDAPP\client>
```

>npm run dev to run the application

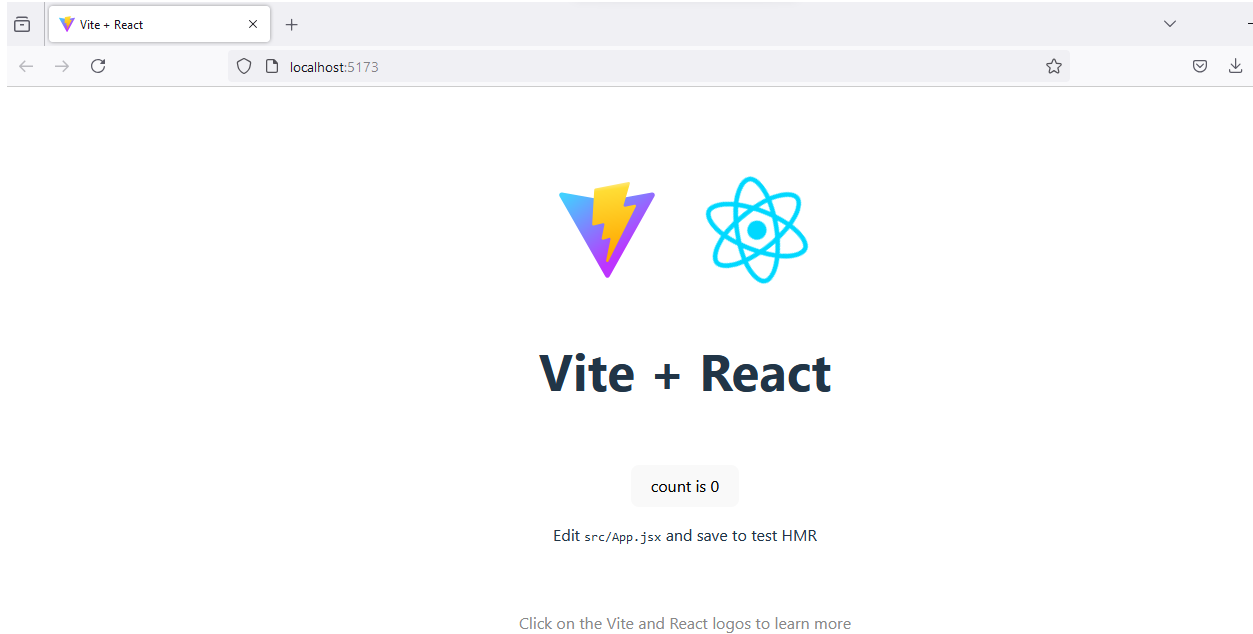
```
PS C:\mycode\CRUDAPP\client> npm run dev

> client@0.0.0 dev
> vite

VITE v5.4.11 ready in 1945 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

run <http://localhost:5173/> on your browser



We can now install other dependencies:

```
>npm install bootstrap react-router-dom axios
```

This command will install three popular libraries: **Bootstrap**, **React Router DOM**, and **Axios**.

Bootstrap

- A CSS framework for building responsive, mobile-first web designs.
- It provides pre-styled components (e.g., buttons, forms, navbars) and utilities for styling (e.g., spacing, typography, layout) without writing custom CSS from scratch

USE: import 'bootstrap/dist/css/bootstrap.min.css';

React Router DOM

- A library for handling routing in React applications.
- It enables navigation between different pages or views in a single-page application (SPA) without a full-page reload.

USE: Import BrowserRouter, Routes, and Route to define routes in your application:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
```

Axios

- A JavaScript library for making HTTP requests.

- Axios simplifies the process of sending requests to APIs and handling responses. Supports GET, POST, PUT, DELETE, and other HTTP methods, making it versatile for interacting with APIs.

USE: Import Axios and make API requests:

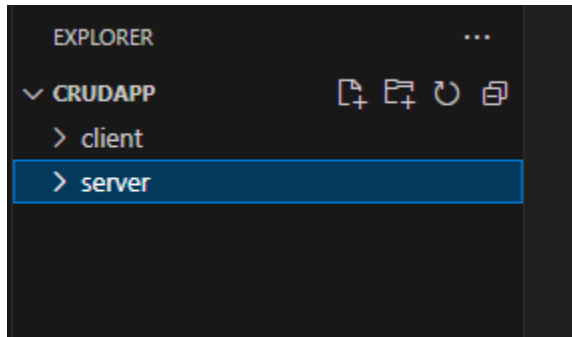
```
import axios from 'axios';
```

```
PS C:\mycode\CRUDAPP\client> npm install bootstrap react-router-dom axios
added 17 packages, and audited 266 packages in 18s

105 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\mycode\CRUDAPP\client>
```

Create a new folder:



Change to server folder:

```
found 0 vulnerabilities
PS C:\mycode\CRUDAPP\client> cd ..
PS C:\mycode\CRUDAPP> cd server
PS C:\mycode\CRUDAPP\server>
```

To create a json file

```

PS C:\mycode\CRUDAPP\server> npm init -y
Wrote to C:\mycode\CRUDAPP\server\package.json:

{
  "name": "server",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

PS C:\mycode\CRUDAPP\server>

```

npm

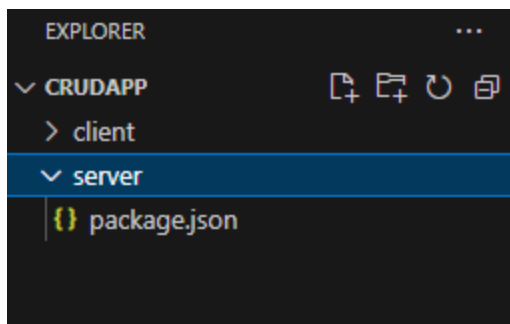
- Node Package Manager, used to manage libraries and dependencies in JavaScript projects.

init

- Initializes a new Node.js project by creating a package.json file.

-y

- Automatically answers "yes" to all configuration prompts, skipping manual input.



Next we install express mysql and cors

1. **Express** is a fast and lightweight web framework for Node.js used to build APIs and web servers. In a React application with a backend, Express handles HTTP requests and responses, such as GET, POST, PUT, and DELETE. It creates the backend server to handle API requests and responses.
2. **MySQL**: Connects the backend to a MySQL database to manage data.

3. **CORS:** Enables secure communication between the frontend (React) and backend (Express) when they are on different origins.

By using these packages together, you create a full-stack application where:

- **React** (frontend) interacts with **Express** (backend).
- **Express** queries the **MySQL** database.
- **CORS** ensures secure cross-origin communication.

NB:

Node.js provides the core functionality (HTTP server, file handling, etc.).

Express.js is built on top of Node.js to streamline building web servers and APIs.

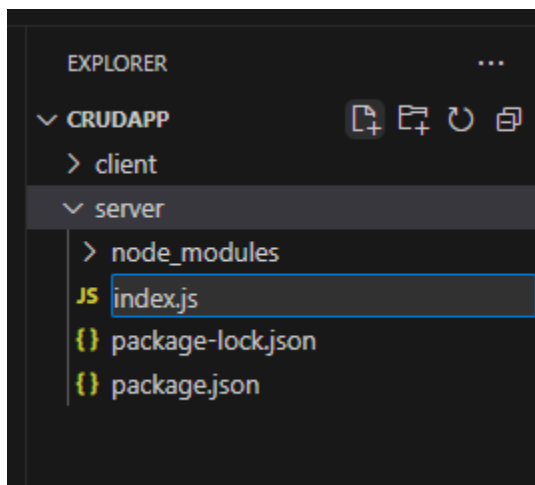
```
PS C:\mycode\CRUDAPP\server> npm install express mysql cors

added 79 packages, and audited 80 packages in 29s

13 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\mycode\CRUDAPP\server> |
```

Create index.js file inside the server folder



Install nodemon (optional)

nodemon is a tool that helps develop Node.js based applications by automatically restarting the node application when file changes in the directory are detected. Install an application to auto refresh changes made while coding


```
PS C:\mycode\CRUDAPP\server> npm install nodemon

added 29 packages, and audited 109 packages in 12s

17 packages are looking for funding
  run `npm fund` for details

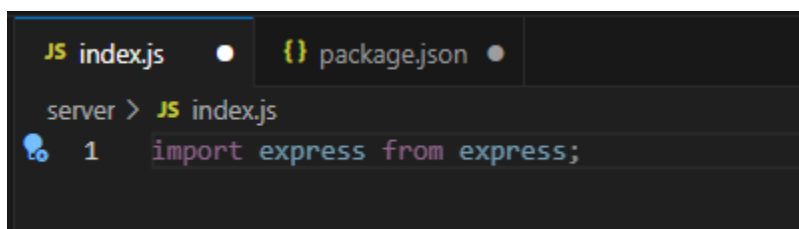
found 0 vulnerabilities
PS C:\mycode\CRUDAPP\server> 
```

Goto package.json and modify as:

```
{
  "name": "server",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
```

To

```
{
  "name": "server",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1" ,
    "start": "nodemon index.js"
  },
```



```
JS index.js  {} package.json

server > JS index.js
1  import express from express;
```

index.js

```
import express from 'express'
import mysql from 'mysql'
import cors from 'cors'

const app=express()
app.use(express.json())
app.use(cors())

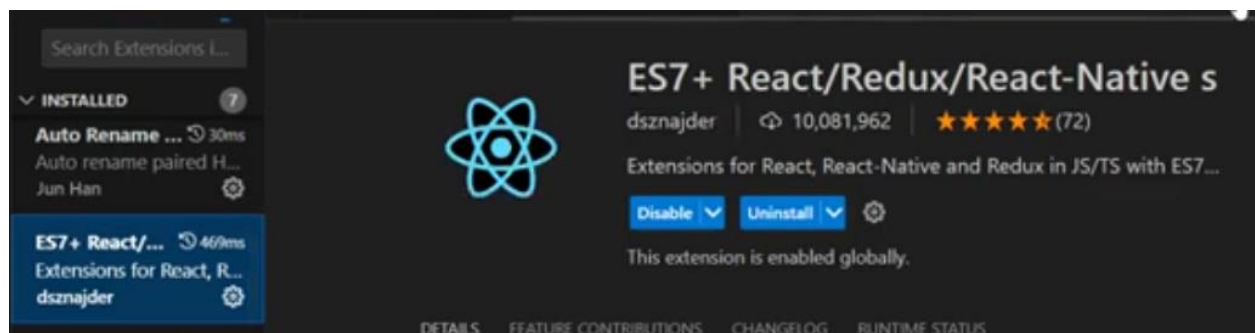
app.listen(3030,()=>{
  console.log("Running")
})
```

```
PS C:\mycode\CRUDAPP\server> npm start

> server@1.0.0 start
> nodemon index.js

[nodemon] 3.1.7
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Running
```

Optional extension that you would want to add to your project



NB: **ES7+ React/Redux/React-Native snippets**" specifically refers to a popular **Visual Studio Code extension** designed to enhance productivity when working with React, Redux, and React Native projects by providing **code snippets**. It also supports modern JavaScript syntax (ES7+).

GO TO The Project and Modify the `server/index.js`

```
import express from "express";
import mysql from "mysql";
import cors from "cors";
const app = express()
app.use(express.json())
app.use(cors())
```

//setting up database connection

```
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "crud",
  dateStrings: 'date'
})
```

//Setting api for retrieving/viewing data from a table

```
app.get('/', (req, res) => {
  const sql = "select * from student";
  db.query(sql, (err, data) => {
    if (err) {
      return res.json({ Error: "Error" })
    }
    return res.json(data)
  })
})
```

//adding/saving a record to a table through post method

```
app.post('/create', (req, res) => {
  const sql = "insert into student (firstname,lastname,date)values (?)"
```

```
const values = [
  req.body.firstname,
  req.body.lastname,
  req.body.date
]
db.query(sql, [values], (err, data) => {
  if (err) {
    return res.json({ Error: "Error" })
  }
  return res.json(data)
})
})
```

[//editing/updating a record in a table through put](#)

```
app.put('/update/:id', (req, res) => {
  const sql = "update student set firstname=?, lastname=?, date=?
where id=?";
  const values = [
    req.body.firstname,
    req.body.lastname,
    req.body.date
  ]
  const id = req.params.id;
  db.query(sql, [...values, id], (err, data) => {
    if (err) {
      return res.json({ Error: "Error" })
    }
    return res.json(data)
  })
})
```

[//delete a record from a table](#)

```
app.delete('/delete/:id', (req, res) => {
```

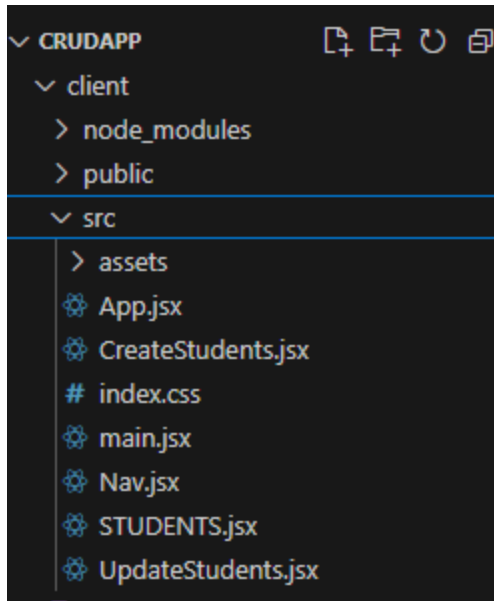
```
const sql = "delete from student where id=?";
const id = req.params.id;
db.query(sql, [id], (err, data) => {
  if (err) {
    return res.json({ Error: "Error" })
  }
  return res.json(data)
})
})
```

[//search/select a specific record from table](#)

```
app.get('/getrecord/:id', (req, res) => {
  const id = req.params.id;
  const sql = "select * from student where id=?"
  db.query(sql, [id], (err, data) => {
    if (err) {
      return res.json({ Error: "Error" })
    }
    return res.json(data)
  })
})
```

```
app.listen(3030, () => {
  console.log("Running")
})
```

CLIENT SIDE



- We will create files CreateStudent.jsx, Nav.jsx, Student.jsx, UpdateStudents.jsx
- We will modify App.jsx

NB: In a React application, App.jsx (or App.js) serves as the main or root component of the application. It acts as the entry point where the core structure and logic of the application are defined.

Default App.jsx

```
import React from 'react'
```

```
function App() {  
  return (  
    <div>App</div>  
  )  
}
```

```
export default App
```

Modified App.jsx

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'
```

```
import Nav from './Nav'
```

```
import Students from './STUDENTS'
```

```
import CreateStudents from './CreateStudents'
```

```
import UpdateStudents from './UpdateStudents'
```

```
function App() {
```

```
  return (
```

```
    <BrowserRouter>
```

```
      <Nav />
```

```
      <Routes>
```

```
        <Route index element={<Students />}></Route>
```

```
        <Route path='/create' element={<CreateStudents />}></Route>
```

```
        <Route path='/update/:id' element={<UpdateStudents  
/>}></Route>
```

```
      </Routes>
```

```
    </BrowserRouter>
```

```
  )
```

```
}
```

```
export default App;
```

Viewing Records

Students.jsx

```
import React, { useEffect, useState } from "react";
import axios from "axios";
import { Link } from "react-router-dom";

const Students = () => {
  const [students, setStudents] = useState([])
  useEffect(() => {
    axios.get('http://localhost:3030')
      .then(res => setStudents(res.data))
      .catch(err => console.log(err))
  }, [])
  const handleDelete=(id)=>{
    axios.delete('http://localhost:3030/delete/'+id)
      .then(res=>window.location.reload())
      .catch(err=>console.log(err))
  }
  return (
    <div className="container mt-5">
      <Link to="/create" className="btn btn-success">Add
Student</Link>
      {students.length !== 0 ? //strapping the table
        <table className="table">
          <thead>
            <tr>
              <th scope="col">#</th>
              <th scope="col">First Name</th>
              <th scope="col">Last Name</th>
```



```

        <th scope="col">Date</th>
        <th scope="col">Action</th>
      </tr>
    </thead>
    <tbody>
      {
        students.map(student =>
          <tr key={student.id}>
            <td>{student.id}</td>
            <td>{student.firstname}</td>
            <td>{student.lastname}</td>
            <td>{student.date}</td>
            <td>
              <Link to={` /update/${student.id}`} className="btn btn-info
btn-sm">Update</Link>
              <button type="button"
onClick={()=>handleDelete(student.id)} className="btn btn-danger
btn-sm">Delete</button>
            </td>
          </tr>
        )
      }
    </tbody>
  </table>
  : <h2>No Records</h2>
}

</div>
)
}

```

```

export default Students

```

Student Management System

Add Student

No Records

NB: Once records have been added to database, No Records will be replaced by a list of records from the table as shown below:

Student Management System

Add Student

#	First Name	Last Name	Date	Action	
2	Fredy	John	2024-11-24	Update	Delete
3	Anne	Jake	2024-11-12	Update	Delete
5	Jonathan	Fredrico	2024-11-11	Update	Delete

Adding Records

Student Management System

Add Student Records

First Name:

Last Name:

Date:



CreateStudent.jsx

```
import axios from "axios";
import React, { useState } from "react";
import { useNavigate } from "react-router-dom";

const CreateStudents = () => {
  const [values, setValues] = useState({
    firstname: "",
    lastname: "",
    date: ""
  })
}
```

```
const navigate=useNavigate()
```

```
const handleSubmit = (e) => {
```

```
  e.preventDefault()
```

```
  //sends data to the backend server for processing,
```

```
  axios.post("http://localhost:3030/create", values)
```

```
    .then(res => navigate('/'))
```

```
    .catch(err => console.log(err))
```

```
}
```

```
return (
```

```
  <div className="d-flex align-items-center flex-column mt-3">
```

```
    <h1>Add Student Records</h1>
```

```
    <form className="w-50" onSubmit={handleSubmit}>
```

```
      <div className="mb-3 mt-3">
```

```
        <label htmlFor="firstname" className="form-label">
```

```
          First Name:
```

```
        </label>
```

```
        <input type="text"
```

```
          className="form-control"
```

```
          id="firstname"
```

```
          placeholder="Enter First Name"
```

```
        name="firstname"
        onChange={(e) => setValues({ ...values, firstname: e.target.value
    }}}
    />
```

```
</div>
<div className="mb-3">
    <label htmlFor="lastname" className="form-label">
        Last Name:
    </label>
    <input type="text"
        className="form-control"
        placeholder="Enter Surname"
        name="lastname"
        onChange={(e) => setValues({ ...values, lastname: e.target.value
    }}}
    />
```

```
</div>
<div className="mb-3">
    <label htmlFor="date" className="form-label">
        Date:
    </label>
    <input type="date"
```

```
        className="form-control"
        name="date"
        onChange={(e) => setValues({ ...values, date: e.target.value })}
      />
    </div>

    <button type="submit" className="btn btn-
primary">Submit</button>
  </form>
</div>
)
}
```

```
export default CreateStudents
```

UpdateStudents.jsx

```
import axios from "axios";
import React, { useEffect, useState } from "react";
import { useNavigate, useParams } from "react-router-dom";

const UpdateStudents = () => {
```

```
const { id } = useParams()
const [values, setValues] = useState({
  firstname: "",
  lastname: "",
  date: ""
})
```

```
const navigate = useNavigate()
```

```
const handleSubmit = (e) => {
  e.preventDefault()
  axios.put("http://localhost:3030/update/"+id, values)
    .then(res => navigate("/"))
    .catch(err => console.log(err))
}
```

```
useEffect(() => {
  axios
    .get("http://localhost:3030/getrecord/" + id)
    .then(res =>
      setValues({
        ...values,
        firstname: res.data[0].firstname,
```

```

        lastname: res.data[0].lastname,
        date: res.data[0].date
    })
)
    .catch(err => console.log(err));
}, []);
return (
    <div className="d-flex align-items-center flex-column mt-3">
        <h1>Update Student Records</h1>
        <form className="w-50" onSubmit={handleSubmit}>
            <div className="mb-3 mt-3">
                <label htmlFor="firstname" className="form-label">
                    First Name:
                </label>
                <input type="text"
                    className="form-control"
                    id="firstname"
                    placeholder="Enter First Name"
                    name="firstname"
                    value={values.firstname}
                    onChange={(e) => setValues({ ...values, firstname: e.target.value
}})

```



```
/>
```

```
</div>
```

```
<div className="mb-3">
```

```
  <label htmlFor="lastname" className="form-label">
```

```
    Last Name:
```

```
  </label>
```

```
  <input type="text"
```

```
    className="form-control"
```

```
    placeholder="Enter Surname"
```

```
    name="lastname"
```

```
    value={values.lastname}
```

```
    onChange={(e) => setValues({ ...values, lastname: e.target.value
```

```
  }}}</div>
```

```
/>
```

```
</div>
```

```
<div className="mb-3">
```

```
  <label htmlFor="date" className="form-label">
```

```
    Date:
```

```
  </label>
```

```
  <input type="date"
```

```
    className="form-control"
```

```
    name="date"
    value={values.date} //assign original value from record
    onChange={(e) => setValues({ ...values, date: e.target.value })}
  />
</div>

  <button type="submit" className="btn btn-
primary">Submit</button>
</form>
</div>
)
}
```

```
export default UpdateStudents
```